ХАКАТОН «ОТКРОЙ МОСПРОМ» · КЕЙС LINQ

LinQ Safety

Техническая документация

Видеоконтроль нарушений техники безопасности на российском ускорителе LinQ TPU

Кейс	хакатон «Открой Моспром», кейс LinQ
Версия	0.1.0 · ветка `main`, 24.10.2026
Запуск	`docker compose up -d --build` → http://localhost:8080
Стек	Python 3.12, FastAPI, Redis Streams, PostgreSQL 16, MinIO, MediaMTX, React 18 + Ant Design 5, Docker Compose
Нейросети	YOLOv8n-pose (люди и 17 точек скелета), YOLOv8s-objects (погрузчик, рохля, телефон) — INT8 на LinQ TPU, SDK 2026.9.45
Данные для обучения	LOCO (склады, public domain), COCO 2017
Время	в интерфейсе, журнале и CSV — московское (`APP_TZ=Europe/Moscow`); в очередях — unix-время, в API — ISO 8601 с поясом
Координаты	рамки, ключевые точки и полигоны зон — доли кадра 0..1, от разрешения камеры не зависят

Оглавление

1. Коротко о решении
2. Ключевые возможности
3. Словарь: термины простыми словами
4. Архитектура
5. Инференс на LinQ TPU
6. Нейросети: подготовка и качество
7. Анализ: от детекций к одному оповещению
8. События, оповещения и роли
9. Руководство пользователя: экран за экраном
10. Производительность и проверка качества
11. API
12. Установка и запуск
13. Структура репозитория
14. Технологии и библиотеки
15. Ограничения и развитие
16. Приложения

1. Коротко о решении

1.1. Какую задачу решаем

На производстве камер больше, чем людей, которые могут на них смотреть. Человек заходит в опасную зону пресса, проходит в метре от погрузчика, идёт по цеху с телефоном у уха — и это видно на записи, но никто не видит этого в ту минуту, когда ещё можно вмешаться.

Решение смотрит на все камеры сразу и за секунды сообщает диспетчеру и мастеру участка: **что** нарушено, **где** (камера, зона), **когда**, и прикладывает **доказательство** — кадр с рамкой и короткий клип. Требования кейса, вокруг которых построена система:

- нейросети считаются на российском ускорителе **LinQ TPU**, а скорость измерена и показана;
- **одно нарушение — одно оповещение**: случайный кадр тревогу не поднимает, повтор того же нарушения не дублирует её;
- журнал с фильтрами по типу, камере, участку, зоне и времени;
- новый тип нарушения добавляется без переделки системы.

1.2. Что делает решение

- **Принимает видео** из трёх источников: RTSP-камеры, видеофайлы (проигрываются по кругу в реальном времени), папки с кадрами. Камера добавляется из интерфейса и подхватывается без перезапуска.
- **Распознаёт на LinQ TPU** две нейросети на каждом кадре одновременно: YOLOv8n-pose находит людей и их скелеты, YOLOv8s-objects — погрузчики, рохли и телефоны. Обе квантованы в INT8 и скомпилированы SDK LinQ.
- **Решает, когда поднять тревогу**: трекинг людей (ByteTrack), сценарии-плагины, конечный автомат подтверждения — нарушение должно длиться заданное время, а не мелькнуть на одном кадре.
- **Сохраняет доказательства**: исходный кадр с хэшем SHA-256, кадр с рамкой и зоной, клип MP4 (H.264) от 5 с до начала нарушения до 5 с после подтверждения.
- **Оповещает** в браузере по WebSocket со звуком — только тех, кому событие положено по роли и участку; дублирует в Telegram; эскалирует, если событие не приняли за 2–60 минут в зависимости от важности.
- **Ведёт журнал**: статусы, история действий (только добавление), фильтры, экспорт CSV, дашборд «где и когда нарушают», онлайн-оценка точности по отметкам операторов.
- **Настраивается без кода**: полигоны зон рисуются на кадре камеры, правила включаются с параметрами, расписанием и сроком действия; форма параметров строится по JSON Schema сценария.
- **Измеряет себя**: метрики Prometheus во всех сервисах, дашборд Grafana, страница «Производительность» с бенчмарком и калькулятором «сколько камер потянет устройство».
- **Не падает целиком**: очереди с подтверждением обработки, очередь «мёртвых» сообщений, аварийный переход на CPU, горячий путь не зависит от сервисов входа и настройки.

1.3. Чего решение не делает

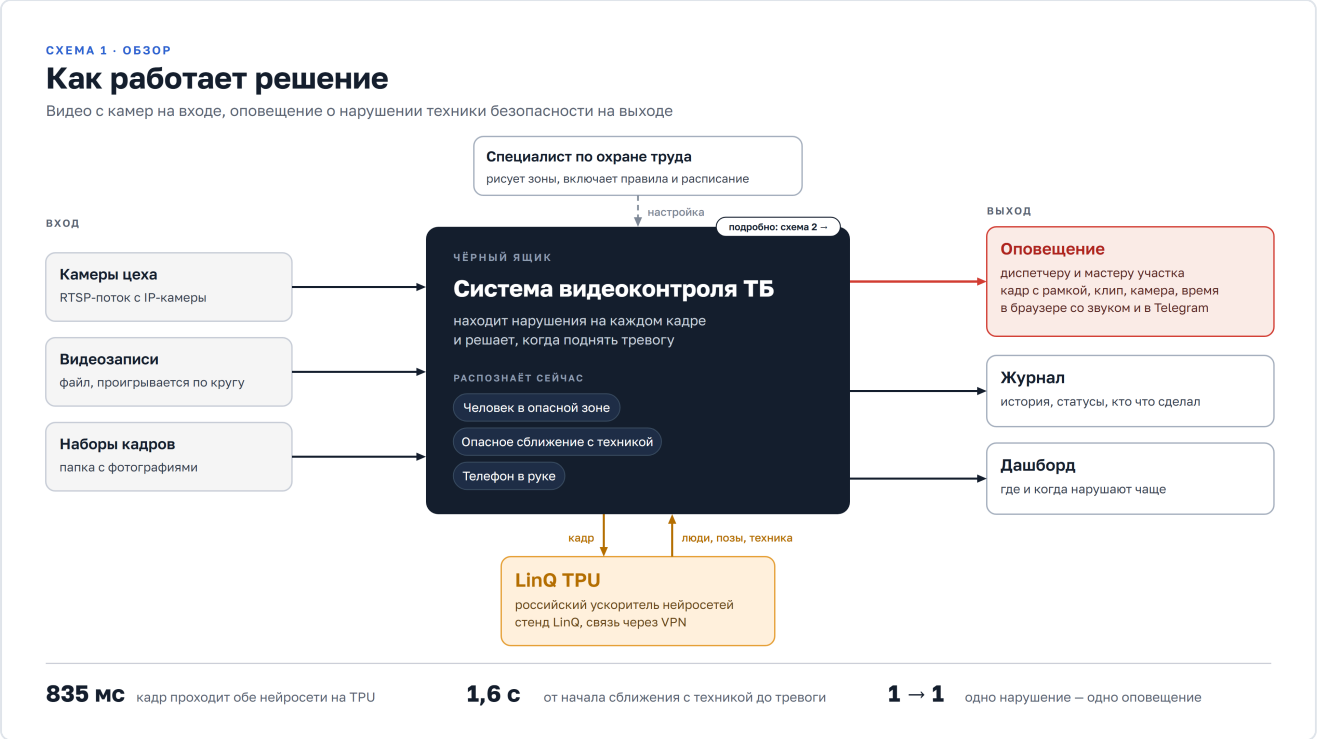
Перечисляем прямо, чтобы это не пришлось выяснять на защите.

- **Распознаёт три типа нарушений, а не восемь.** В замысле (docs/ARCHITECTURE.md §7.2) были ещё падение, бег, курение, перелезание и работа на высоте без привязи. Реализованы и проверены на

TPU: человек в опасной зоне, опасное сближение с техникой, телефон в руке. Кейс прямо говорит, что точность важнее числа сценариев; каркас для остальных готов — сценарий добавляется одним файлом (§7.8).

- **Не узнаёт людей.** Ни распознавания лиц, ни персональных данных: только рамки и скелеты. Номер трека живёт, пока человек в кадре.
- **Не решает за человека.** Событие остаётся «Новым», пока его не примет оператор; подтверждение, отметка «ложное» и «устранено» — действия людей, каждое пишется в историю.
- **Не мерит расстояния в метрах.** Опасная дистанция задаётся в ростах человека в кадре — порог не зависит от того, насколько далеко камера, но и не требует калибровки сцены.
- **Событийные метрики на размеченных роликах не измерены.** Precision и recall на уровне событий, ложные тревоги в час — в плане, но набора размеченных роликов с производства у нас нет (§10.7). Качество моделей измерено на отложенной выборке, совпадение TPU с CPU — на реальных кадрах.

1.4. Решение за одну минуту



Для показа нужны две команды: `docker compose up -d --build` поднимает всю систему, браузер на http://localhost:8080 открывает интерфейс. Без роликов и без доступа к стенду система тоже работает: контейнер `cams` крутит в RTSP тестовую таблицу, а детектор `synthetic` водит по кадру «человека» и «погрузчик» — так весь путь от кадра до оповещения проверяется на любой машине. Боевой режим — `INFERENCE_BACKEND=broker`: кадры уходят на TPU стенда LinQ через VPN.

1.5. Режимы инференса

Код сервиса один, меняется переменная `INFERENCE_BACKEND`. Всё остальное — трекинг, сценарии, журнал, интерфейс — не знает, где посчитана нейросеть.

Режим	Где считается	Зачем	Состояние
`broker`	**LinQ TPU стенда** через RabbitMQ и воркеры в Jupyter	основной режим	работает, замерен (§10)

Режим	Где считается	Зачем	Состояние
`onnx`	CPU этой машины, те же модели в ONNX Runtime	разработка и аварийный режим	работает
`synthetic`	без нейросети: «человек» и «погрузчик» ходят по кадру	проверка конвейера и интерфейса без моделей	работает
`null`	пустые детекции	нагрузочная проверка без нейросети	работает
`remote`	HTTP-воркер `POST /detect` на стенде	если стенд даст открыть порт	клиент написан, на этом стенде не применим
`pytpu`	прямой доступ к `/dev/tpu*`	если TPU стоит в той же машине	на стенде Jupyter не видит устройство; бэкенд не реализован

`INFERENCE_FALLBACK=onnx` включает аварийный режим: если основной бэкенд трижды подряд ошибся, кадры 30 секунд идут на CPU, затем снова на TPU. Активный бэкенд виден в `GET /api/inference/models`, на странице «Производительность» и в метрике `inference_fallback_active`.

2. Ключевые возможности

#	Возможность	Что это даёт	Где в решении
1	Две нейросети на TPU за один кадр	люди со скелетами и техника приходят вместе; обе программы отправляются на TPU одновременно, а не по очереди	`backend/ml/worker/linq_worker.py`
2	Разрез графа: тело на TPU, хвост на CPU	TPU считает свёрточное тело в INT8, декодирование рамок и NMS — небольшой ONNX-хвост; совпадение с полной моделью проверяется	`backend/ml/export/cut_head.py`, `verify_split.py`
3	Калибровка с отсечением выбросов	у модели техники корреляция логитов TPU и float выросла с 0,65 до 0,98 — без этого TPU не видел погрузчик	`backend/ml/compile/compile_tpu.py`
4	Свежий кадр важнее полного	кадр старше 2 с не считается; кадр камеры, у которой предыдущий ещё на TPU, пропускается — очередь не копится	`services/inference/app/main.py`
5	Аварийный CPU-режим	стенд недоступен — те же модели в ONNX Runtime, через 30 с снова TPU	`services/inference/detectors/fallback.py`
6	VPN только в контейнере	сеть хоста не трогаем; RabbitMQ стенда виден остальным сервисам как `linq-vpn:5672`	`backend/server/infra/linq-vpn/`
7	Сценарии-плагины	новый тип нарушения — один файл; форма параметров в интерфейсе строится по его JSON Schema	`services/analytics/scenarios/`
8	Конечный автомат подтверждения	случайный кадр не поднимает тревогу; одно нарушение — ровно одно событие; повтор во время паузы подавлен	`services/analytics/engine/fsm.py`
9	Склейка треков	человек скрылся за погрузчиком и вернулся с новым номером — нарушение продолжается, второго события нет	`services/analytics/engine/engine.py`
10	Состояние переживает рестарт	активные нарушения сохраняются в Redis каждые 2 с: после перезапуска событие закроется, а не задвоится	`Engine.snapshot` / `restore`
11	Время по меткам кадров	автомат и трекер считают время по кадрам, а не по часам сервера: прогон видео быстрее реального времени даёт тот же результат	`engine/fsm.py`, `tools/replay.py`
12	Зоны, правила и расписание без кода	полигон на кадре, параметры правила, дни недели и интервал, «до 12:00 сегодня»	`services/camera`, `frontend/src/pages/CameraSetup.tsx`
13	Горячая перезагрузка конфигурации	правка зоны применяется к следующему кадру; горячий путь читает конфигурацию из Redis и работает, даже если сервис настройки лежит	`libs/common/common/camconfig.py`

#	Возможность	Что это даёт	Где в решении
14	Доказательная база	исходный кадр с SHA-256, кадр с рамкой, клип ±5 с; история действий только дополняется	`services/analytics/app/evidence.py`, `services/events`
15	Оповещение по роли и участку	мастер видит только свой участок; подписки на типы, участки и минимальную важность — в личном кабинете	`services/notifications`
16	Одна сводка вместо стопки плашек	непринятые события собираются в одну сводку с числом критичных и последним событием	`frontend/src/components/AppLayout.tsx`
17	Эскалация	непринятое событие через 2 / 5 / 15 / 60 минут (по важности) уходит руководству, охране труда и мастеру	`services/events/app/main.py`
18	Telegram	кадр с рамкой и подпись — в чат мастера; эскалации — всем ответственным	`services/notifications/app/main.py`
19	Онлайн-точность	доля подтверждённых среди отмеченных операторами — по сценариям и камерам	`GET /api/events/stats/precision`
20	Дашборд	динамика, типы, тепловая карта «день недели × час», камеры, участки; клик по столбцу — журнал с фильтром	`frontend/src/pages/Dashboard.tsx`
21	Шесть ролей	диспетчер, мастер, охрана труда, руководитель, юрист, администратор; права проверяет каждый сервис	`libs/common/common/auth.py`
22	Наблюдаемость	метрики Prometheus во всех сервисах, готовый дашборд Grafana, бенчмарк из интерфейса	`infra/grafana`, `/metrics`
23	Очереди без потерь	Redis Streams с группами: сообщение снимается только после обработки; трижды упавшее уходит в `dlq:*`	`libs/common/common/bus.py`
24	Офлайн-прогон ролика	видео → события с таймкодами без Docker и Redis; детекции можно сохранить и перебирать пороги без нейросети	`backend/server/tools/replay.py`
25	Сквозная проверка	вход → камеры → событие по WebSocket → снапшот → смена статуса → статистика → клип	`backend/server/tools/smoke.py`
26	Интерфейс без бэкенда	`npm run dev:mock` — те же экраны на MSW-моках, новые события каждые ~10 с	`frontend/src/mocks/`

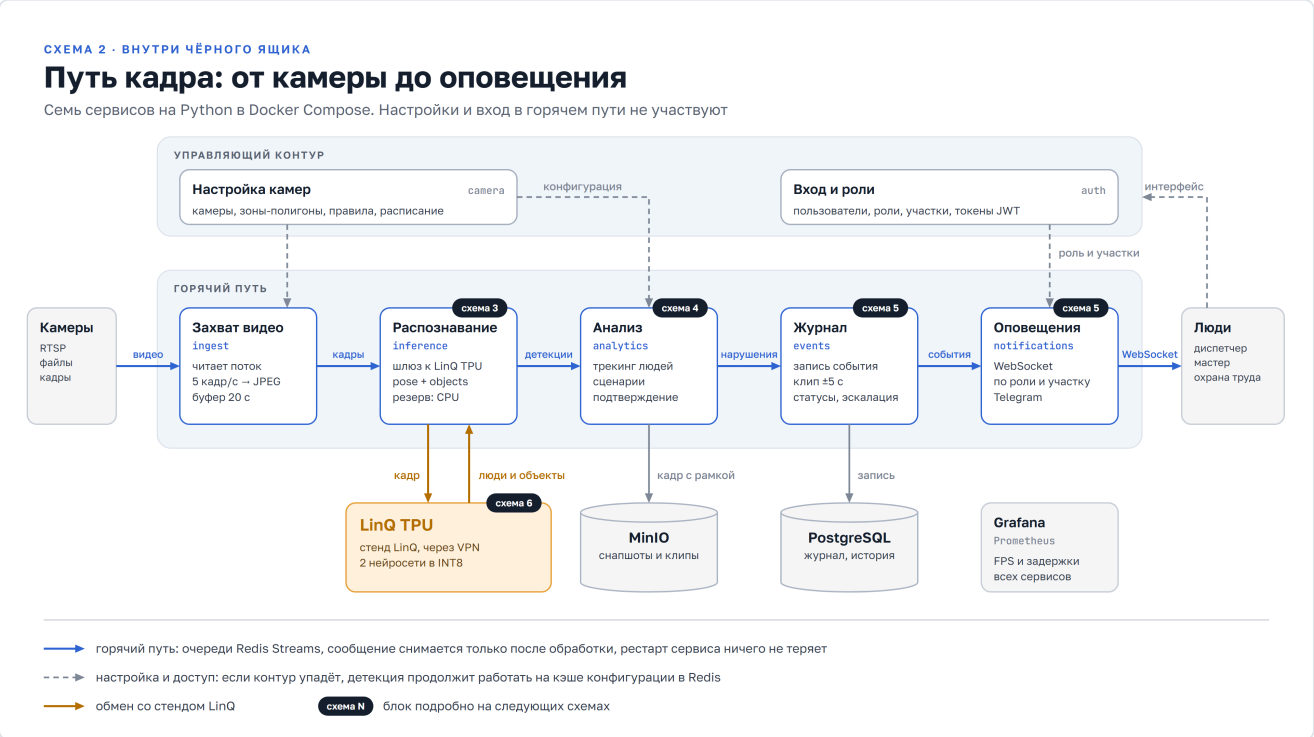
3. Словарь: термины простыми словами

Термин	Что это значит
LinQ TPU	российский ускоритель нейросетей. На стенде хакатона — общие для всех команд устройства, к которым обращаются через очередь RabbitMQ
Инференс	расчёт нейросети на одном кадре: кадр на входе, рамки и точки на выходе
ONNX	открытый формат нейросети; из него SDK LinQ делает программу для TPU, а ONNX Runtime считает ту же сеть на CPU
Программа `.tpu`	скомпилированная под TPU сеть; лежит на стенде в `/auto/cache/<логин>/`
INT8, квантование	перевод весов и активаций из 32-битных чисел в 8-битные. TPU считает в INT8; рантайм сам переводит float32 ↔ int8
Калибровка	прогон реальных кадров через сеть, чтобы выбрать диапазоны квантования для каждого слоя
Перцентиль активаций	при калибровке берётся не максимум, а 99,9-й перцентиль: редкие выбросы не растягивают шкалу и не съедают точность
Тело и хвост сети	граф разрезан: тело (свёртки) считает TPU, хвост (декодирование рамок, сигмоида) — CPU
Letterbox	вписывание кадра в квадрат 640×640 с серыми полями без искажения пропорций
NMS	подавление дублей: из пересекающихся рамок одного объекта остаётся самая уверенная
DFL	способ, которым YOLOv8 кодирует границы рамки распределением; декодируется в хвосте сети
Поза, ключевые точки	17 точек скелета COCO: нос, глаза, уши, плечи, локти, кисти, бёдра, колени, лодыжки
Точка опоры	середина между лодыжками, а если их не видно — середина нижней стороны рамки человека. По ней решается «в зоне или нет»
Трек, ByteTrack	один и тот же человек от кадра к кадру под одним номером. ByteTrack — алгоритм сопоставления рамок
Сценарий	правило распознавания одного типа нарушения, плагин аналитики. На каждом кадре даёт оценку 0..1 для каждого человека
Правило	сценарий, включённый на конкретной камере: с зоной или без, с параметрами, важностью и расписанием
Зона	полигон на кадре камеры: опасная, запретная, «высота», ограждение, «не курить»
Конечный автомат	состояния нарушения: нет → кандидат → активно → пауза. Решает, когда оповещать и когда нарушение кончилось
Окно подтверждения (`confirm_s`)	сколько секунд нарушение должно длиться, чтобы стать событием
Пауза (`cooldown_s`)	сколько секунд после конца нарушения повтор того же правила по тому же человеку не оповещает
Склейка треков	если трек оборвался и рядом появился новый, нарушение переходит к новому треку, а не начинается заново

Термин	Что это значит
Горячий путь	цепочка «кадр → детекции → нарушение → событие → оповещение». Остальное — управляющий контур
Redis Streams	очереди в Redis. Группа читателей (consumer group) получает каждое сообщение один раз, подтверждение (ack) — после обработки
DLQ	очередь «мёртвых» сообщений `dlq:<поток>`: туда уходит то, что трижды не удалось обработать
RabbitMQ, `reply_to`	брокер очередей стенда LinQ. Запрос несёт адрес временной очереди для ответа и `correlation_id` — номер запроса
RTSP, HLS	RTSP — протокол IP-камер; HLS — видео в браузере. MediaMTX переводит одно в другое
JWT, JWKS	JWT — подписанный токен входа; JWKS — публичный ключ, по которому каждый сервис проверяет подпись сам
Эскалация	событие не приняли вовремя — оно уходит выше по ролям
p50 / p95	медиана и 95-й перцентиль: половина замеров не больше p50, 95 % — не больше p95
Сквозная задержка	от кадра, подтвердившего нарушение, до отправки оповещения в браузер

4. Архитектура

4.1. Общая схема



Семь собственных сервисов на Python и FastAPI, веб-интерфейс на React и инфраструктура — всё в одном `docker-compose.yml`. Сервисы делятся на два контура:

- **горячий путь** — `ingest → inference → analytics → events → notifications`: кадры, детекции, нарушения и события идут через очереди Redis Streams;
- **управляющий контур** — `auth` (вход, роли, профиль) и `camera` (камеры, зоны, правила): обычные REST-сервисы с базой. Горячий путь берёт у них только данные, разложенные в Redis, поэтому продолжает находить нарушения, даже если оба упадут.

TPU физически стоит на стенде LinQ. Наш сервер говорит с ним через контейнер `linq-vpn` и очередь RabbitMQ стенда (§5).

4.2. Принципы

1. **Горячий путь отделён от управляющего.** Вход, настройки и статистика не участвуют в доставке тревоги.
2. **Сервис владеет своими данными.** У `auth`, `camera` и `events` свои схемы PostgreSQL; в чужую схему никто не ходит — только API или события в Redis.
3. **Железо за абстракцией.** `Detector` в инференсе (synthetic / onnx / broker / remote), `Scenario` в аналитике, источник в захвате (RTSP / файл / папка). Смена TPU на CPU — одна переменная окружения.
4. **Ни одно сообщение не теряется молча.** Подтверждение после обработки, повтор, DLQ; повторная доставка безопасна — запись события идемпотентна по его `event_id`.
5. **Свежесть важнее полноты.** Для тревоги нужен текущий кадр: устаревшие и лишние кадры пропускаются и считаются в метриках, а не копятся в очереди.

6. **Время — по кадрам.** Подтверждение, окончание нарушения и трекинг считаются по меткам времени кадров, поэтому офлайн-прогон ролика и живой поток дают одинаковые события.
7. **Наблюдаемость с первого дня.** Каждый сервис отдаёт `/health` и `/metrics`; задержки на TPU — обязательная часть сдачи.
8. **Всё запускается одной командой**, кроме воркеров на стенде LinQ — они запускаются одним скриптом (§5.7).

4.3. Сервисы

Сервис	Адрес за шлюзом	Что делает	Хранит
`auth`	`/api/auth`	пользователи и роли, вход, JWT RS256 (access 15 мин, одноразовый refresh 7 дней), профиль, подписки на уведомления	PostgreSQL `auth`
`camera`	`/api/config`	площадки → участки → камеры, зоны-полигоны, правила с параметрами и расписанием, снапшот камеры, каталог сценариев	PostgreSQL `config` → Redis
`ingest`	`/api/ingest`	поток на каждую камеру: RTSP / файл / папка кадров, прореживание до FPS анализа, JPEG, кольцевой буфер 20 с, сборка клипов MP4	Redis (кадры), MinIO (клипы)
`inference`	`/api/inference`	шлюз к TPU: кадры → детекции; аварийный режим, бенчмарк	Redis
`analytics`	`/api/analytics`	трекинг, сценарии, конечный автомат, доказательства	Redis (состояние), MinIO (кадры)
`events`	`/api/events`	журнал, статусы и история, статистика, CSV, эскалация, приём нарушений по HTTP	PostgreSQL `events`
`notifications`	`/ws`, `/api/notifications`	WebSocket с фильтром по роли, участку и подпискам; Telegram	—
`frontend`	`/`	SPA на React (nginx)	—

Инфраструктура: Traefik 3.1 (шлюз, порт 8080), PostgreSQL 16, Redis 7 (AOF, `noeviction`), MinIO, MediaMTX 1.9 (RTSP → HLS/WebRTC), Prometheus 2.54, Grafana 11.2, контейнер `cams` (имитация камер из роликов) и `linq-vpn` (доступ к стенду, профиль `linq`).

4.4. Горячий путь: потоки и сообщения

Поток Redis	Пишет	Читает (группа)	Что внутри	Длина
`frames:{shard}`	`ingest`	`inference`	ссылка на кадр; сам JPEG лежит в ключе `frame:{cam}:{seq}` 60 с	~500
`detections`	`inference`	`analytics`	люди со скелетами и объекты кадра, задержки по стадиям	~2000
`violations`	`analytics`, `POST /ingest/violations`	`events`	начало (`started`) и конец (`ended`) нарушения	~10 000
`clips:requests`	`events`	`ingest`	запрос клипа за интервал	~10 000

Поток Redis	Пишет	Читает (группа)	Что внутри	Длина
`clips:ready`	`ingest`	`events`	клип собран или не собран	~10 000

Шард потока кадров выбирается стабильным хэшем камеры (`crc32 % FRAME_SHARDS`): кадры одной камеры всегда идут по порядку в одну очередь, а инференс можно масштабировать по шардам. Контракты — Pydantic-модели в `backend/server/libs/common/common/messages.py`:

```
// frames:{shard} — FrameMsg
{"camera_id": "cam-03", "seq": 18234, "ts": 1792868002.06, "frame_key": "frame:cam-03:18234", "w": 1280, "h": 720}

// detections — DetectionsMsg
{"camera_id": "cam-03", "seq": 18234, "ts": 1792868002.06, "frame_key": "frame:cam-03:18234", "w": 1280, "h": 720,
 "backend": "linq-tpu",
 "models": {"pose": "pose@yolov8n-pose-v8.4.0-linq-tpu", "objects": "objects@yolov8s-objects-v8.4.0-linq-tpu"},
 "latency_ms": {"queue": 3, "pre": 21, "infer": 721, "post": 43, "rpc": 161, "total": 950},
 "persons": [{"bbox": [0.80, 0.30, 0.85, 0.58], "score": 0.81, "kpts": [[0.82, 0.32, 0.97], "…17 точек"]}],
 "objects": [{"cls": "forklift", "bbox": [0.55, 0.30, 0.70, 0.62], "score": 0.64}]}

// violations — ViolationMsg (у пары started/ended общий event_id)
{"event_id": "723ef739-…", "kind": "started", "camera_id": "cam-03", "camera_name": "Камера 3", "area_id": 2,
 "zone_id": null, "rule_id": 4, "scenario": "proximity", "scenario_title": "Опасное сближение с техникой",
 "severity": "critical", "track_id": 42, "started_at": 1792867459.1, "confirmed_at": 1792867460.3,
 "snapshot_key": "events/2026/10/24/723ef739-….jpg", "snapshot_sha256": "9f2c…",
 "annotated_key": "events/2026/10/24/723ef739-…_annotated.jpg",
 "bbox": [0.80, 0.30, 0.85, 0.58], "confidence": 1.0, "frame_ts": 1792867460.3, "seq": 18234}
```

4.5. Каналы и ключи Redis

Ключ / канал	Тип	Кто пишет → кто читает	Зачем
`config:cameras`	hash	`camera` → `ingest`, `analytics`, `events`	полная конфигурация каждой камеры с зонами и правилами
`config.updated`	Pub/Sub	`camera` → `ingest`, `analytics`	«перечитай камеру» (плюс сверка раз в 30 с на случай потерянного сообщения)
`events.created` / `events.updated` / `events.escalated`	Pub/Sub	`events` → `notifications`	событие для WebSocket и Telegram
`frame:{cam}:{seq}`	string, TTL 60 с	`ingest` → `inference`, `analytics`	JPEG кадра
`frame:last:{cam}`	string, TTL 600 с	`ingest` → `camera`, бенчмарк	последний кадр — фон редактора зон
`ingest:status:{cam}`	hash, TTL 30 с	`ingest` → `camera`	состояние потока: в сети, FPS, ошибка
`scenarios:catalog`	JSON	`analytics` → `camera`, интерфейс	загруженные сценарии с JSON Schema параметров
`analytics:state`	JSON	`analytics` → `analytics`	активные нарушения и паузы — переживают рестарт
`inference:status`	JSON, TTL 30 с	`inference`	активный бэкенд, аварийный режим, FPS
`inference:benchmarks`	list (50)	`inference`	история замеров

Ключ / канал	Тип	Кто пишет → кто читает	Зачем
`notify:users`	hash	`auth` → `notifications`	роль, участки, подписки и Telegram каждого пользователя
`dlq:{поток}`	stream	шина	сообщения, которые не удалось обработать трижды

4.6. Хранилища

PostgreSQL — по схеме на сервис, таблицы создаются при старте (`create_all`, без миграций — осознанное упрощение хакатона):

Схема	Таблицы	Что важно
`auth`	`users`, `refresh_tokens`	пароли — argon2; refresh-токен хранится только хэшем SHA-256 и одноразовый
`config`	`sites`, `areas`, `cameras`, `zones`, `rules`	полигоны, параметры и расписания — JSONB; у камеры `version` растёт при каждой правке
`events`	`events`, `event_actions`	`event_actions` только дополняется: кто, что, когда, из какого статуса в какой, комментарий

Событие хранит имя камеры, зоны и сценария **на момент нарушения**: переименование камеры не переписывает историю.

MinIO (бакет `linq`, чтение анонимное — ключи содержат UUID события, поэтому `<img>` и `<video>` работают через шлюз `/media/linq/…` без подписанных ссылок):

Ключ	Что
`events/ГГГГ/ММ/ДД/<event_id>.jpg`	исходный кадр, подтвердивший нарушение (его SHA-256 — в журнале)
`events/ГГГГ/ММ/ДД/<event_id>_annotated.jpg`	тот же кадр с рамкой человека и полигоном зоны
`clips/ГГГГ/ММ/ДД/<event_id>.mp4`	клип H.264, ±5 с вокруг нарушения

4.7. Надёжность: что будет, если…

Что случилось	Что происходит
Упал любой сервис горячего пути	неподтверждённые сообщения остались в группе: после рестарта сервис дочитывает свои, зависшие у упавшего экземпляра забираются через `XAUTOCLAIM` (30 с)
Сообщение ломает обработчик	три попытки с паузой, затем — в `dlq:<поток>` с причиной; поток не встаёт
Перезапустили `analytics` во время нарушения	состояние автоматов поднимается из `analytics:state`: человек ещё в кадре — событие продолжается, ушёл — придёт `ended`, второго оповещения нет
Повторная доставка нарушения	`INSERT … ON CONFLICT DO NOTHING` по `event_id`: событие не задваивается
Лежат `camera`, `auth` или PostgreSQL	детекция и оповещения работают на конфигурации из Redis; не работают вход новых пользователей и правка настроек

Что случилось	Что происходит
Стенд LinQ не отвечает	три ошибки подряд → 30 с на CPU (`INFERENCE_FALLBACK=onnx`) → снова TPU
Камера отвалилась	поток переподключается с нарастающей паузой до 30 с (сразу, если только что работал); статус «не в сети» виден в интерфейсе
Медленный браузер	очередь клиента 200 сообщений; переполнилась — соединение закрывается (код 1013), клиент переподключается сам
Фоновая задача сервиса упала	`Supervisor` перезапускает её с паузой 1 → 30 с
Клип не собрался за 3 минуты	статус клипа `failed` , событие и кадры остаются

4.8. Шлюз

Traefik публикует всё на одном порту (по умолчанию 8080). Префикс `/api/<сервис>` срезается, а Swagger каждого сервиса открывается по `/api/<сервис>/docs` .

Путь	Куда
`/api/auth` , `/api/config` , `/api/events` , `/api/inference` , `/api/ingest` , `/api/analytics` , `/api/notifications`	соответствующий сервис
`/ws`	WebSocket `notifications`
`/live/<камера>/index.m3u8`	HLS из MediaMTX
`/media/<бакет>/<ключ>`	MinIO: снапшоты и клипы
`/`	SPA (`frontend` , nginx)

5. Инференс на LinQ TPU

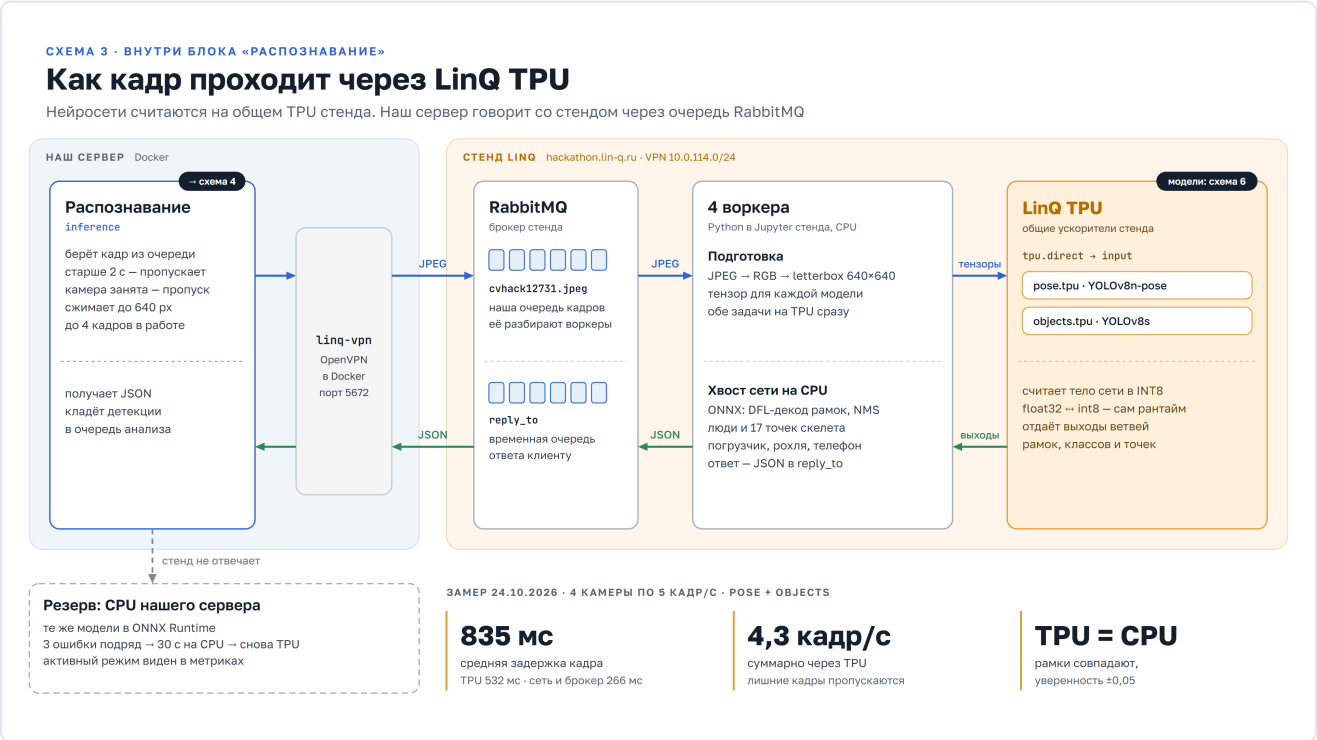
5.1. Как устроен стенд

Это определило архитектуру сильнее всего остального.

- Стенд — JupyterHub на `hackathon.lin-q.ru` (10.0.114.30) в VPN 10.0.114.0/24. В нашем Jupyter **нет устройства** `/dev/tpu*`: TPU — общий облачный сервис стенда.
- Задача для TPU — сообщение `PyTask` в RabbitMQ стенда (exchange `tpu.direct`, ключ `input`) с путём к программе `.tpu`. Программа должна лежать в `/auto/cache/<логин>/`. Ответ приходит в очередь, указанную в заголовке.
- TPU общие для всех команд: **замеры включают ожидание в очереди стенда**, это не чистое время вычисления.
- Сервис TPU **кэширует программы по пути**: перезаписанный файл с тем же именем не подхватывается. Поэтому в имя программы вписан хэш её содержимого (`yolov8s_objects_b1_o1_h1m_1b310853.tpu`).
- Процессы в Jupyter живут, пока работает сервер Jupyter; после его перезапуска воркеры нужно запустить снова.

Отсюда схема: наш сервер не держит TPU-код у себя, а отправляет JPEG в собственную очередь на брокере стенда; воркеры в Jupyter готовят тензоры, отправляют их на TPU и возвращают компактный JSON.

5.2. Путь кадра через TPU



1. `inference` берёт ссылку на кадр из `frames:{shard}`. Кадр старше `MAX_FRAME_AGE_S` (2 с) не считается (`inference_dropped_total{reason="stale"}`).
2. Если у этой камеры предыдущий кадр ещё на TPU или в работе уже `INFERENCE_CONCURRENCY` кадров — кадр пропускается (`reason="busy"`). В модель всегда идёт свежий кадр, очередь не копится.

3. Кадр крупнее 640 px уменьшается до отправки: воркер всё равно вписывает его в 640×640, а лишние байты нагружают VPN.
4. JPEG уходит в очередь `cvhack12731.jpeg` на брокере стенда через `linq-vpn:5672`. Соединения с брокером держатся открытыми и переиспользуются: рукопожатие AMQP через VPN стоит несколько обходов сети.
5. Один из четырёх воркеров в Jupyter декодирует JPEG, делает letterbox 640×640 и тензоры для **обеих** моделей и отправляет на TPU **две задачи сразу**, каждую со своей очередью ответа.
6. TPU возвращает выходы ветвей тела сети. Воркер прогоняет их через ONNX-хвосты на CPU (декодирование рамок, сигмоида, ключевые точки), делает NMS и переводит координаты в доли исходного кадра.
7. JSON с людьми, объектами и задержками по стадиям возвращается в `reply_to` клиента; `inference` добавляет время сети (`rpc`) и кладёт детекции в поток `detections`.

5.3. Протокол обмена

Участок	Что передаётся
`inference` → наша очередь	тело — JPEG; `reply_to` — временная эксклюзивная очередь клиента; `correlation_id` — номер запроса; `expiration` = таймаут (3 с): опоздавший запрос брокер удалит сам; заголовки `camera_id` , `ts`
воркер → TPU	`PyTask(data={"images": тензор 1×3×640×640})` в `tpu.direct/input` ; заголовки `TARGET_URI = file:///auto/cache/<логин>/<программа>.tpu?raw=False` , `RESPONSE_ROUTING_KEYS` — очередь ответа, `MESSAGE_ID`
воркер → `inference`	JSON в формате `DetectResult` (`persons` , `objects` , `backend` , `models` , `latency_ms`) или `{"error": "…"}`

Воркер обрабатывает кадры последовательно (`prefetch_count=1` , подтверждение после ответа), ограничивает JPEG 8 МБ, ждёт TPU до 10 с. Если клиент уже ушёл по таймауту и его очереди нет — ответ молча отбрасывается. Опоздавший ответ на старый запрос клиент тоже пропускает: он сверяет `correlation_id`.

5.4. Аварийный режим

`FallbackDetector` оборачивает основной детектор и запасной. Три ошибки подряд (таймаут, обрыв соединения, ошибка воркера) переключают кадры на запасной на 30 с; затем снова пробуется основной. Запасной — те же модели в ONNX Runtime на CPU из `data/models/<имя>/` , поэтому сценарии и пороги не меняются. Детекции в аварийном режиме помечены бэкендом `onnx (аварийный)` , а в Prometheus поднимается `inference_fallback_active`.

5.5. Доступ к стенду из Docker

VPN поднимается **только внутри контейнера** `linq-vpn` (OpenVPN, профиль compose `linq`) — сеть хоста не меняется. Контейнер:

- пробрасывает RabbitMQ стенда остальным сервисам как `linq-vpn:5672` (`socat` , переменная `LINQ_FORWARDS`);
- содержит утилиту `ljup` для работы с JupyterHub без браузера: `login` , `ls` , `get` , `put` , `py` (код в ядре), `sh` (команда в терминале), `kernels` , `interrupt` , `restart` ;
- берёт логин и пароль VPN из `secrets/vpn-auth.txt` , учётку Jupyter — из `secrets/jupyter.env` ; токен API и номер ядра переживают пересоздание контейнера (`secrets/ljup-state.json`). Папка `secrets/`

в git не попадает.

```
docker compose --profile linq up -d linq-vpn
docker compose exec -T linq-vpn ljup login
docker compose exec -T linq-vpn ljup sh 'pgrep -fa linq_worker'      # живы ли воркеры на стенде
```

5.6. Что лежит на стенде

Путь на стенде	Что
`~/work/cvhack_worker/`	бандл: код воркера, `ml_runtime`, `meta.json` и ONNX-хвосты моделей (`backend/ml/worker/package.py`)
`/auto/cache/<логин>/*.tpu`	программы TPU: `yolov8n_pose_b1_o1_h1m.tpu`, `yolov8s_objects_b1_o1_h1m_1b310853.tpu`
`~/work/cvhack_worker/logs/worker-N.log`	логи воркеров

Дополнительно в Jupyter ставятся `onnxruntime==1.29.0` и `opencv-python-headless==4.11.0.86` (`pip install --user`); `pika` и клиент формата задач TPU `tpu_pycloud_format` в окружении стенда уже есть.

5.7. Запуск воркеров

```
docker compose exec -T linq-vpn ljup sh 'sh ~/work/cvhack_worker/backend/ml/worker/start_workers.sh'
```

Скрипт копирует недостающие программы в `/auto/cache/$USER/`, останавливает старые воркеры, запускает `WORKERS` (по умолчанию 4) новых в фоне на одну очередь и печатает `Listening on RabbitMQ queue cvhack12731.jpeg` для каждого. Переменная `MODELS` сужает набор моделей (по умолчанию — все, у которых есть `meta.json`). Общий TPU стенда обслуживает запросы параллельно: один воркер с моделью pose даёт около 3 кадров/с, четыре — около 10.

В `.env` нашего сервера:

```
INFERENCE_BACKEND=broker
INFERENCE_FALLBACK=onnx
INFERENCE_CONCURRENCY=4        # до N кадров в полёте, по одному на камеру; во время замеров §10 — 7
```

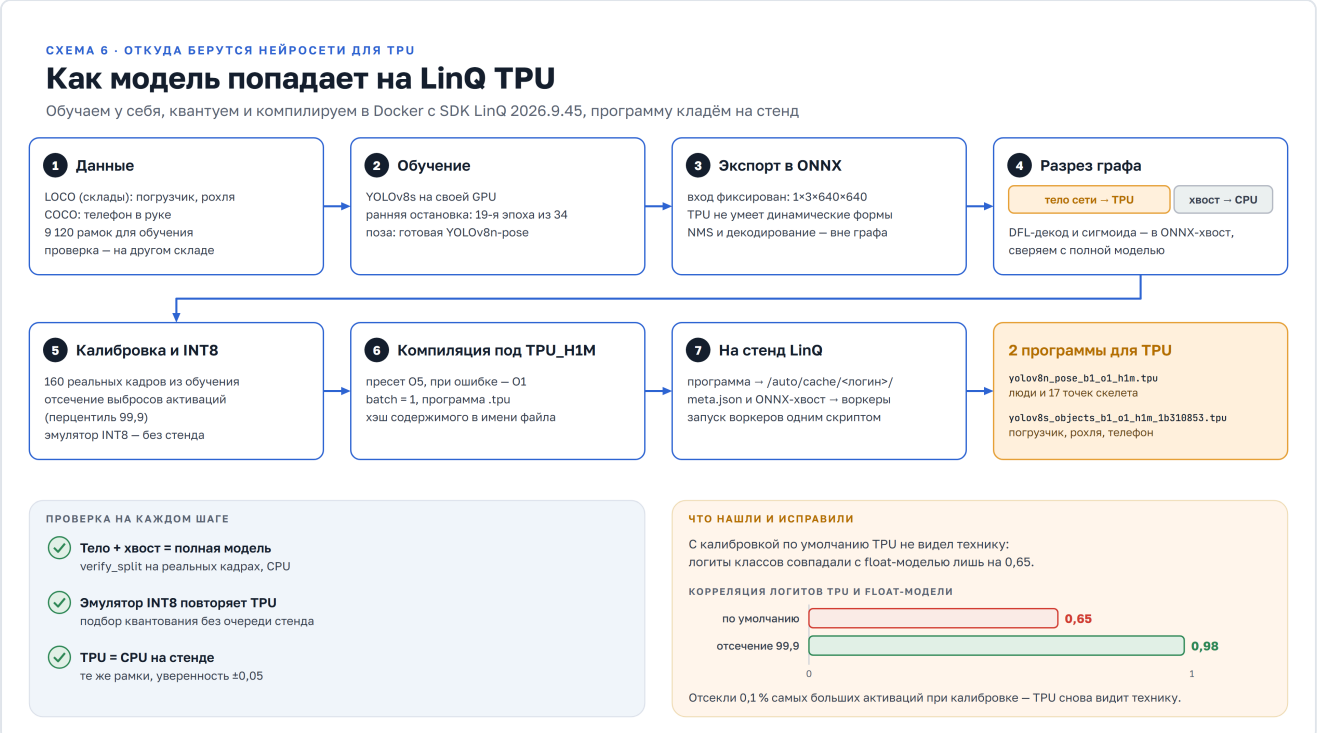
6. Нейросети: подготовка и качество

6.1. Модели

Модель	Что находит	Основа	Вход	Программа TPU	Калибровка
`pose`	люди + 17 точек скелета	YOLOv8n-pose, веса COCO (Ultralytics 8.4.0)	1×3×640×640	`yolov8n_pose_b1_o1_h1m.tpu`, O1	32 кадра COCO128
`objects`	`forklift` (погрузчик), `pallet_truck` (рохля), `phone`	YOLOv8s, дообучена нами	1×3×640×640	`yolov8s_objects_b1_o1_h1m_1b310853.tpu`, O1	160 кадров из обучения, перцентиль 99,9

Обе модели лежат в репозитории в `data/models/<имя>/`: `meta.json`, полная ONNX (для CPU и аварийного режима), ONNX-хвост, программа `.tpu`; у `objects` — ещё веса `.pt`. `meta.json` описывает имена и формы входа и выходов разреза, классы, файлы, пресет компиляции, число калибровочных кадров и хэш исходной модели — по нему сервис и воркер собирают модель без правки кода.

6.2. Как модель попадает на TPU



Компиляция идёт в Docker-образе с официальным SDK LinQ 2026.9.45 (Linux x86_64, Python 3.12.11 — на «голом» Windows SDK не работает). Образ собирается из распакованного архива SDK:

```
docker build --platform linux/amd64 -f backend/ml/compile/Dockerfile -t cvhack-linq:2026.9.45 /path/to/tpu_framework
RUN="docker run --rm --platform linux/amd64 -v $PWD:/work cvhack-linq:2026.9.45"

$RUN python backend/ml/export/export_onnx.py data/runs/objects/weights/best.pt data/models/objects/yolov8s-objects.onnx
$RUN python backend/ml/export/cut_head.py data/models/objects/yolov8s-objects.onnx --out-dir data/models/objects --name
objects
$RUN python backend/ml/export/verify_split.py data/models/objects data/calib/objects --limit 20
$RUN python backend/ml/compile/compile_tpu.py data/models/objects data/calib/objects --limit 160 --activation-percentile
99.9
```

Шаг	Что делает	Почему так
Экспорт ONNX	статичный вход 1×3×640×640, opset 12, без NMS, без упрощения графа	TPU не поддерживает динамические формы; упрощение в Ultralytics 8.4.0 удаляет имена узлов, по которым режется граф
Разрез графа	тело — до узлов `/model.22/Concat_output_0`, `Concat_1` (и `Concat_2` у позы); хвост — всё после	свёрточное тело считает TPU, декодирование DFL, сигмоида и ключевые точки — маленький ONNX-хвост на CPU
Проверка разреза	тело + хвост против полной модели на реальных кадрах: число детекций совпадает, расхождение выходов ≤ 1e-4	ошибка в разрезе ловится до квантования, а не на стенде
Калибровка	`RegularModel.calibrate_torch` на реальных кадрах с той же предобработкой, что в рантайме	SDK в примере калибрует на случайном шуме — так нельзя
Квантование	INT8, пороги сохраняются в `thresholds.json`	TPU считает в INT8
Компиляция	`compiler.compile_(TPU_H1M, …)`: сначала O5, при ошибке — O1; batch 1	итоговый пресет обеих программ — O1
Имя программы	хэш содержимого в имени файла, обновление `meta.json`	сервис TPU кэширует программы по пути

6.3. Квантование: история с погрузчиком

С калибровкой по умолчанию модель техники на TPU не находила погрузчик там, где его видел CPU. Логиты классов TPU коррелировали с float-моделью всего на **0,65**: несколько экстремальных активаций растягивали шкалу квантования, и полезный сигнал терялся в младших битах. Калибровка по 99,9-му перцентилю активаций (`--activation-percentile 99.9`) отсекла выбросы — корреляция стала **0,98**, уверенности на TPU лишь немного ниже, чем на CPU.

Перебирать настройки удобно без стенда и его очереди: эмулятор SDK `QuantizedModel.load(qm).as_torch_graph().gm(round(x·127))` воспроизводит выходы TPU на CPU.

6.4. Данные и обучение модели `objects`

Класс	Источник	Рамок в обучении	Рамок в проверке
`forklift`	LOCO, склады, public domain	479 (кадры повторены ×4)	119
`pallet_truck`	LOCO	2 268	559
`phone`	COCO 2017, класс `cell phone`	6 373	261

- Для проверки из LOCO отложен целый `subset-4` — другой склад. Соседние почти одинаковые кадры видео не попадают одновременно в обучение и проверку.

- Погрузчиков в LOCO мало (598 рамок против 2 827 у рохлей), поэтому кадры с ними повторены четыре раза.
- Кадры без целевых объектов — 10 % от позитивных; картинки заранее уменьшены до 640 px.
- Обучение: YOLOv8s с весами COCO, 640 px, batch 12, AdamW lr 0,001, косинусное расписание, ранняя остановка (терпение 15 эпох) — лучшая эпоха 19 из 34. Без смешанной точности: на RTX 5060 Ti с torch 2.11 fp16 разгонял ошибку классификации до сотен и обнулял mAP.
- Сборка набора — `backend/ml/datasets/build_objects.py`, обучение — `backend/ml/training/train_objects.py`.

6.5. Качество моделей

`objects` на отложенной выборке (другой склад LOCO и фото COCO):

Класс	Precision	Recall	mAP50
`forklift`	0,24	0,51	0,36
`pallet_truck`	0,61	0,20	0,28
`phone`	0,56	0,52	0,53

Модель путает погрузчик и рохлю между собой, но для сценария сближения это один класс «техника». Детектор на камерах сверху находит технику не на каждом кадре — поэтому сценарий помнит её `memory_s` секунд (§7.3).

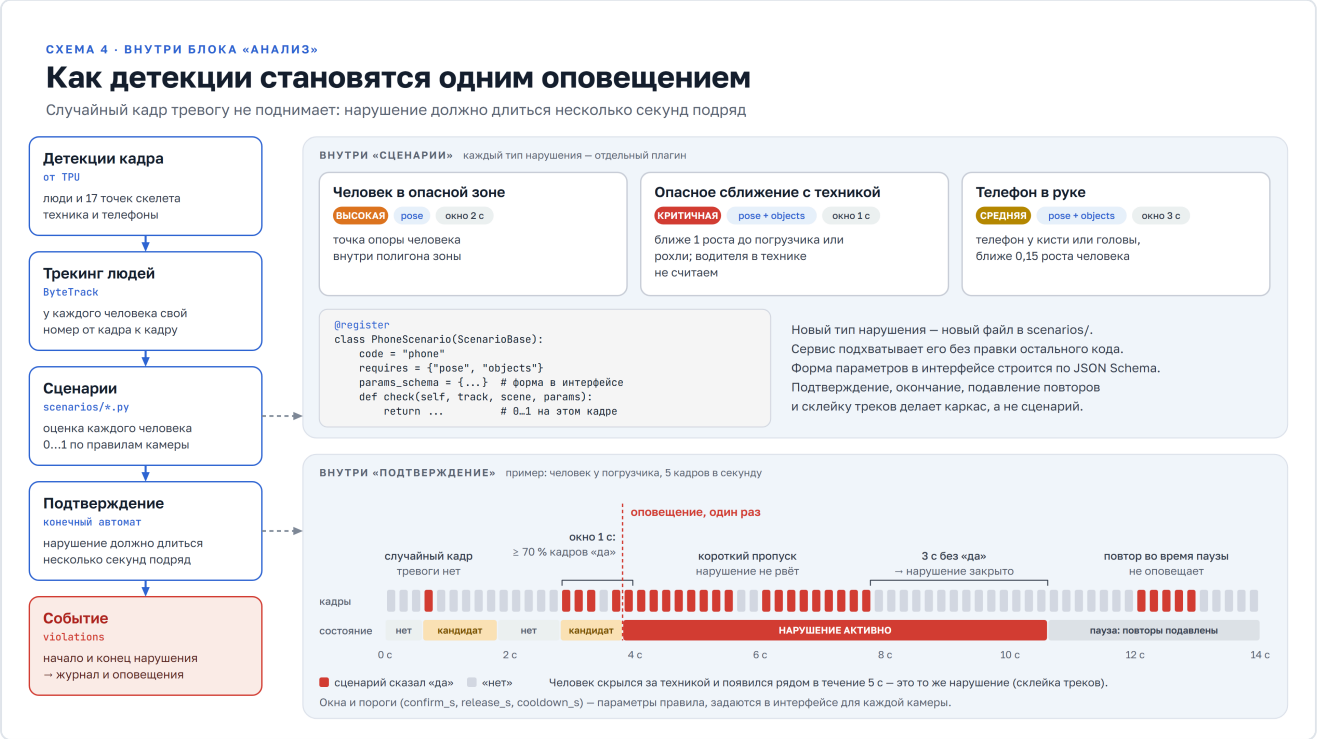
`pose` — готовая YOLOv8n-pose. Первый запуск на TPU: на проверочном кадре CPU и TPU нашли одного и того же человека с уверенностью 0,862 и 0,861. Модель пропускает мелких и далёких людей: на пешеходном ролике из сэмплов OpenCV она находит 2–3 человека из 7 — на CPU столько же, то есть дело в модели, а не в TPU.

TPU против CPU. На живом конвейере TPU и ONNX Runtime на CPU дают совпадающие рамки и уверенности (±0,05). Собственный постпроцессинг детекции (`postprocess_detect`) совпадает с Ultralytics до рамки: 35 кадров, 204 детекции.

6.6. Почему именно эти модели

Вопрос	Рассмотрели	Выбрали	Почему
Семейство	YOLO11, DAMO-YOLO, RT-DETR	YOLOv8	`yolov8n_pose` есть в списке моделей, поддержанных SDK; разрез головы и компиляция проверены на v8
Размер позы	YOLOv8s/m-pose	YOLOv8n-pose	готовая поддержанная сеть, работала на TPU с первого дня; крупнее — следующий шаг (§15)
Техника и телефон	готовые COCO-классы `truck`, `cell phone`	дообучение YOLOv8s	в COCO нет погрузчиков и рохлей
Одна модель или две	одна сеть на все классы	две сети параллельно	поза нужна всем сценариям; модель техники меняется независимо; обе задачи уходят на TPU одновременно
Трекер	DeepSORT	ByteTrack	без отдельной сети внешнего вида — весь TPU отдан детекции; трекер на CPU тратит ~5 мс на кадр

7. Анализ: от детекций к одному оповещению



Движок аналитики — код без ввода-вывода (`engine/engine.py`): один и тот же класс `Engine` работает в сервисе на очередях Redis и в офлайн-прогоне видео `tools/replay.py` .

7.1. Трекинг

- ByteTrack из пакета `trackers` (Roboflow), в старых версиях — `supervision.ByteTrack` . Время кадра передаётся явно, поэтому неровный FPS (пропуски кадров под нагрузкой) не ломает трекинг.
- Параметры: активация трека — уверенность 0,45, трек подтверждается на 2 кадрах, теряется через 2 с.
- Запасной трекер без зависимостей — жадное сопоставление по IoU плюс близость центров (`TRACKER=iou`).
- Трек хранит 10 с истории: рамки, уверенности, ключевые точки. Отсюда сценарии берут точку опоры, рост, скорость.
- Трек без наблюдений дольше 10 с забывается; кадр с номером не больше уже обработанного игнорируется.

7.2. Сценарии

Сценарий — класс в `services/analytics/scenarios/` . Метод `check(track, scene, params)` вызывается на каждом кадре для каждого видимого человека и возвращает 0..1 — «нарушение есть на этом кадре». Всё, что связано со временем, делает автомат, а не сценарий.

Код	Нарушение	Модели	Важность	Окно	Правило
`in_zone`	Человек в опасной зоне	pose	высокая	2 с	точка опоры человека (или центр рамки) внутри полигона зоны; без зоны — человек в любом месте кадра; детекции слабее 0,4 не считаются

Код	Нарушение	Модели	Важность	Окно	Правило
`proximity`	Опасное сближение с техникой	pose + objects	критичная	1 с	от точки опоры человека до рамки погрузчика или рохли не больше `distance_heights` (1,0) ростов человека; водитель — человек, чья рамка на 60 % внутри рамки техники, — не считается; технику, замеченную за последнюю `memory_s` (1 с), сценарий помнит; с зоной — только внутри неё
`phone`	Телефон в руке	pose + objects	средняя	3 с	центр рамки телефона ближе `near_heights` (0,15) роста человека к кисти, носу или уху; если поза не видна — телефон в верхних 2/3 рамки человека; с зоной — только внутри неё

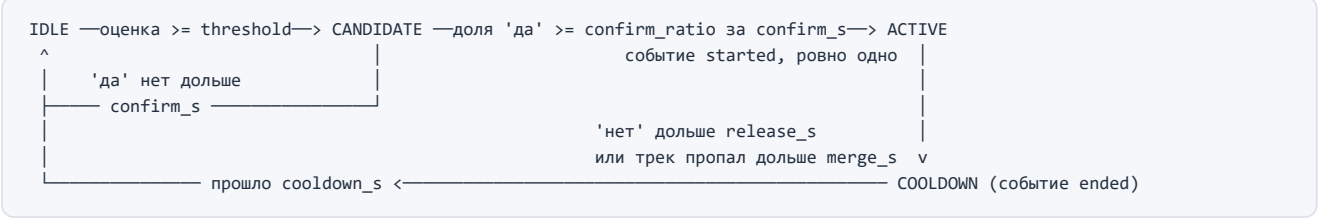
Параметры каждого сценария (видны в интерфейсе как форма):

Сценарий	Параметр	По умолчанию	Смысл
`in_zone`	`point`	`foot`	какая точка человека должна быть в зоне: точка опоры или центр рамки
	`min_score`	0,4	минимальная уверенность детекции человека
`proximity`	`classes`	`forklift` , `pallet_truck`	что считать техникой
	`distance_heights`	1,0	опасное расстояние в ростах человека; 0 — только касание рамки
	`min_score`	0,25	минимальная уверенность детекции техники
	`memory_s`	1,0	сколько секунд учитывать технику после последней детекции
	`driver_overlap`	0,6	какая доля рамки человека внутри техники делает его водителем
`phone`	`near_heights`	0,15	«у кисти или головы», в долях роста
	`min_score`	0,3	минимальная уверенность детекции телефона

Почему расстояние в ростах человека: человек у дальней стены и у камеры занимает в кадре разное число пикселей, а порог «1 рост» одинаково понятен и там, и там — без калибровки камеры и гомографии.

7.3. Конечный автомат подтверждения

Автомат заводится на каждую тройку «камера × правило × трек»:



Параметр	По умолчанию	Что значит
`threshold`	0,5	оценка сценария, с которой кадр считается «да»

Параметр	По умолчанию	Что значит
`confirm_s`	2,0 (у сценариев свои: 2 / 1 / 3)	сколько секунд нарушение должно длиться, чтобы стать событием
`confirm_ratio`	0,7	доля кадров «да» в окне подтверждения: одиночный пропуск детекции не срывает подтверждение
`release_s`	3,0	сколько секунд без «да», чтобы нарушение считалось законченным
`cooldown_s`	30	пауза после конца нарушения: повтор не оповещает
`merge_s`	5	сколько ждать пропавший трек, прежде чем закрыть нарушение
`merge_dist`	0,1	новый трек ближе этого (доля кадра) наследует нарушение пропавшего

Все параметры — значения по умолчанию сценария, их переопределяет правило конкретной камеры. Уверенность события — средняя оценка по кадрам «да» в окне подтверждения.

Что это даёт на практике:

- **случайный кадр тревогу не поднимает** — нужна доля «да» за всё окно;
- **одно нарушение — одно событие**, пока человек в кадре; короткий пропуск детекции внутри активного нарушения его не рвёт;
- **повтор во время паузы не оповещает**;
- **окончание фиксируется**: событию проставляется `ended_at`, длительность видна в журнале.

7.4. Склейка треков и рестарт

Человек скрылся за погрузчиком на пару кадров и вернулся — трекер может дать ему новый номер. Если в течение `merge_s` (5 с) рядом с местом, где пропал трек с активным нарушением (ближе `merge_dist`), появляется новый трек, он **наследует** автомат вместе с `event_id`: второго события нет. Старый трек при этом должен действительно пропасть — если он ещё виден, рядом стоит другой человек.

Тот же механизм закрывает рестарт: активные нарушения и паузы сохраняются в Redis (`analytics:state`) каждые 2 с и при каждом переходе. После перезапуска автоматы поднимаются как «потерявшие трек»: человек ещё в кадре — новый трек унаследует событие, ушёл — через `merge_s` придёт `ended`. Это покрыто тестами `test_restart_restores_active_violation_without_duplicate` и `test_restart_closes_violation_if_person_gone`.

7.5. Расписание и срок действия правил

Правило может действовать по дням недели и в интервале времени (`{"days": [0,1,2,3,4], "from": "07:55", "to": "12:00"}`), в том числе через полночь (`22:00-06:00` — во вторник в 01:00 действует правило понедельника), и до заданного момента (`valid_until`, в интерфейсе — «до 12:00 сегодня»). Время — местное (`APP_TZ`). Правило, которое выключили или у которого кончилось время, закрывает своё активное нарушение событием `ended`.

7.6. Доказательства

При подтверждении нарушения `analytics` берёт из Redis именно тот кадр, который его подтвердил, и сохраняет в MinIO:

- **исходный кадр** и его SHA-256 — хэш пишется в журнал и в CSV, по нему можно проверить, что кадр не подменён;

- **кадр с разметкой** — рамка человека цветом важности, полигон зоны, подпись (на латинице: шрифты OpenCV не умеют кириллицу — русскую подпись поверх кадра рисует интерфейс).

`events` запрашивает у `ingest` клип: от 5 с до начала нарушения до 5 с после подтверждения. `ingest` держит кольцевой буфер последних 20 с каждой камеры (8 кадров/с), дожидается конца окна и собирает MP4 H.264 через ffmpeg (`-movflags +faststart` — клип играет в `<video>` сразу). Оповещение не ждёт клипа: кадр приходит сразу, клип — следом, обновлением события через 5–10 с.

7.7. Офлайн-прогон ролика

```
# синтетический детектор, правила камеры cam-01 из infra/cameras.yml
python backend/server/tools/replay.py data/demo_videos/cam01.mp4 --camera cam-01
# модели на CPU, детекции сохранить — потом перебирать пороги без нейросети
python backend/server/tools/replay.py cam03.mp4 --detector onnx --scenario proximity --save-detections cam03.jsonl
python backend/server/tools/replay.py cam03.mp4 --detections cam03.jsonl --scenario proximity --params '{"distance_heights":
0.7}'
```

Redis, база и Docker не нужны: печатаются события с таймкодами видео. Так подбираются параметры сценариев.

7.8. Как добавить тип нарушения

Новый файл в `backend/server/services/analytics/scenarios/`:

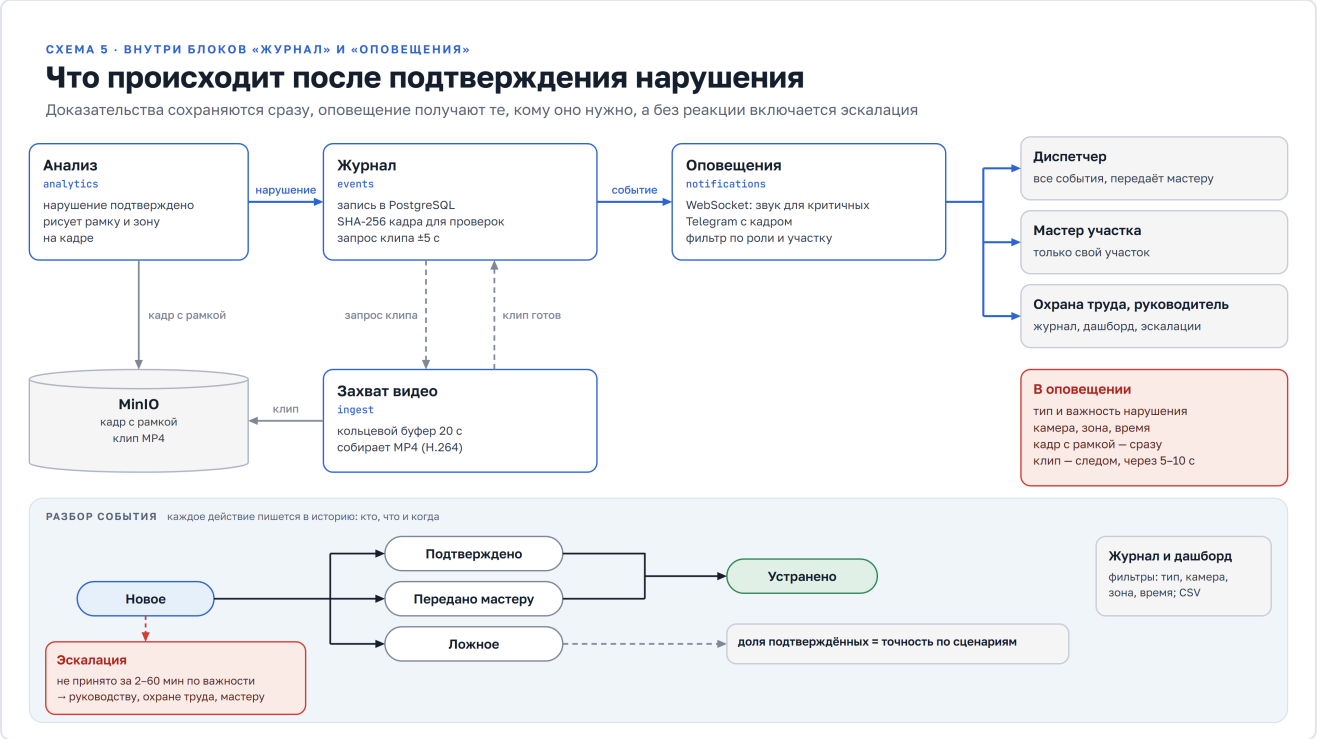
```
from scenarios.base import ScenarioBase, Scene, Severity, Track, register

@register
class PhoneScenario(ScenarioBase):
    code = "phone"
    title = "Телефон в руке"
    requires = {"pose", "objects"}
    severity = Severity.MEDIUM
    params_schema = {"type": "object", "properties": {
        "near_heights": {"type": "number", "default": 0.15, "title": "Телефон у кисти или головы, долей роста"},
        "confirm_s": {"default": 3.0},
    }}

    def check(self, track: Track, scene: Scene, params: dict) -> float:
        ...  # 0..1 — «нарушение есть на этом кадре»
```

Пакет импортирует все модули сам: сценарий сразу появляется в `GET /api/config/scenarios`, форма параметров в интерфейсе строится по `params_schema`, а подтверждение, окончание, паузу и склейку треков делает каркас. Сломанный модуль не роняет сервис — ошибка видна в `GET /api/analytics/scenarios/errors`. Сценарий «на кадр целиком» (огонь, дым) объявляется `per_track = False` и вызывается раз на кадр без трека.

8. События, оповещения и роли



8.1. Жизненный цикл события

Статус	Значит	Куда можно перейти
Новое (`new`)	создано системой, никто не реагировал	подтверждено, ложное, передано мастеру, устранено
Передано мастеру (`forwarded`)	диспетчер передал ответственному	подтверждено, ложное, устранено
Подтверждено (`confirmed`)	нарушение настоящее	устранено, передано мастеру
Ложное (`false`)	система ошиблась	вернуть в новое
Устранено (`resolved`)	меры приняты	вернуть в новое

Недопустимый переход отвергается с кодом 409. Каждое действие — смена статуса, комментарий, назначение, эскалация, окончание, готовность клипа — пишется в `event_actions` с пользователем и временем; таблица только дополняется. Время первой реакции человека сохраняется отдельно — из него дашборд считает «среднее время реакции».

8.2. Эскалация

Фоновая задача `events` раз в 15 секунд ищет события, которые остались «Новыми» дольше порога:

Важность	Порог	Переменная
критичная	2 мин	`ESCALATE_CRITICAL_MIN`
высокая	5 мин	`ESCALATE_HIGH_MIN`
средняя	15 мин	`ESCALATE_MEDIUM_MIN`

Важность	Порог	Переменная
низкая	60 мин	`ESCALATE_LOW_MIN`

Событию ставится отметка «не принято вовремя», в историю пишется действие, по WebSocket уходит `event.escalated`, в Telegram — сообщение администратору, охране труда, руководителю и мастеру участка. Демо-история не эскалируется.

8.3. WebSocket `/ws`

```
// ws://<шлюз>/ws?access_token=<JWT>
{"type": "hello", "user": {"id": 4, "username": "operator", "role": "operator", "area_ids": []}}
{"type": "event.created",   "event": EventOut, "notify": true}   // notify — совпало с подписками пользователя
{"type": "event.updated",   "event": EventOut, "notify": true}   // статус, окончание, готов клип
{"type": "event.escalated", "event": EventOut, "notify": true}   // не принято вовремя
{"type": "ping", "ts": 1792868002.4}                             // раз в 25 с; на "ping" клиента сервер отвечает "pong"
```

- Событие получают только те, кому оно доступно: мастер — по своим участкам, остальные роли — все.
- Поле `notify` говорит, совпало ли событие с подписками из личного кабинета (типы нарушений, участки, минимальная важность): по нему интерфейс решает, играть ли звук и показывать ли всплывающее окно.
- С неверным или истёкшим токеном рукопожатие отклоняется; клиент обновляет токен и переподключается с нарастающей паузой, а при тишине дольше 60 с переподключается сам.
- `EventOut` описан в `messages.py`; координаты нормированы к 0..1.

8.4. Telegram

Если задан `TELEGRAM_BOT_TOKEN`, новое событие уходит пользователям, у которых в личном кабинете указан chat id, включён Telegram и событие проходит подписки; эскалации — всем ответственным ролям. Сообщение — кадр с рамкой и подпись: важность, нарушение, камера, зона, время по Москве.

8.5. Роли и права

Роль	Демо-логин	Стартовый экран	Что может
Диспетчер	`operator`	Камеры	лента всех событий, статусы, «передать мастеру», журнал
Мастер смены	`master`	Камеры	то же, но только события своего участка (цех №1)
Охрана труда	`hse`	Камеры	камеры, зоны, правила, журнал, экспорт, дашборд, производительность
Руководитель	`manager`	Дашборд	дашборд, журнал, экспорт, производительность — только чтение
Юрист / аудитор	`auditor`	Журнал	журнал, дашборд и экспорт — только чтение
Администратор	`admin`	Камеры	всё, включая пользователей и демо-историю

Пароль всех демо-пользователей — `demo` (`DEMO_PASSWORD`). Права проверяет **каждый сервис** по ролям в токене; интерфейс повторяет те же правила, чтобы не показывать недоступные кнопки (`frontend/src/shared/roles.ts`).

Действие	Кто
Смена статуса, комментарий	диспетчер, мастер, охрана труда, администратор
Камеры, зоны, правила	охрана труда, администратор
Экспорт CSV	охрана труда, руководитель, юрист, администратор
Пользователи, демо-история	администратор
Чтение журнала и статистики	любой вошедший (мастер — по своим участкам)

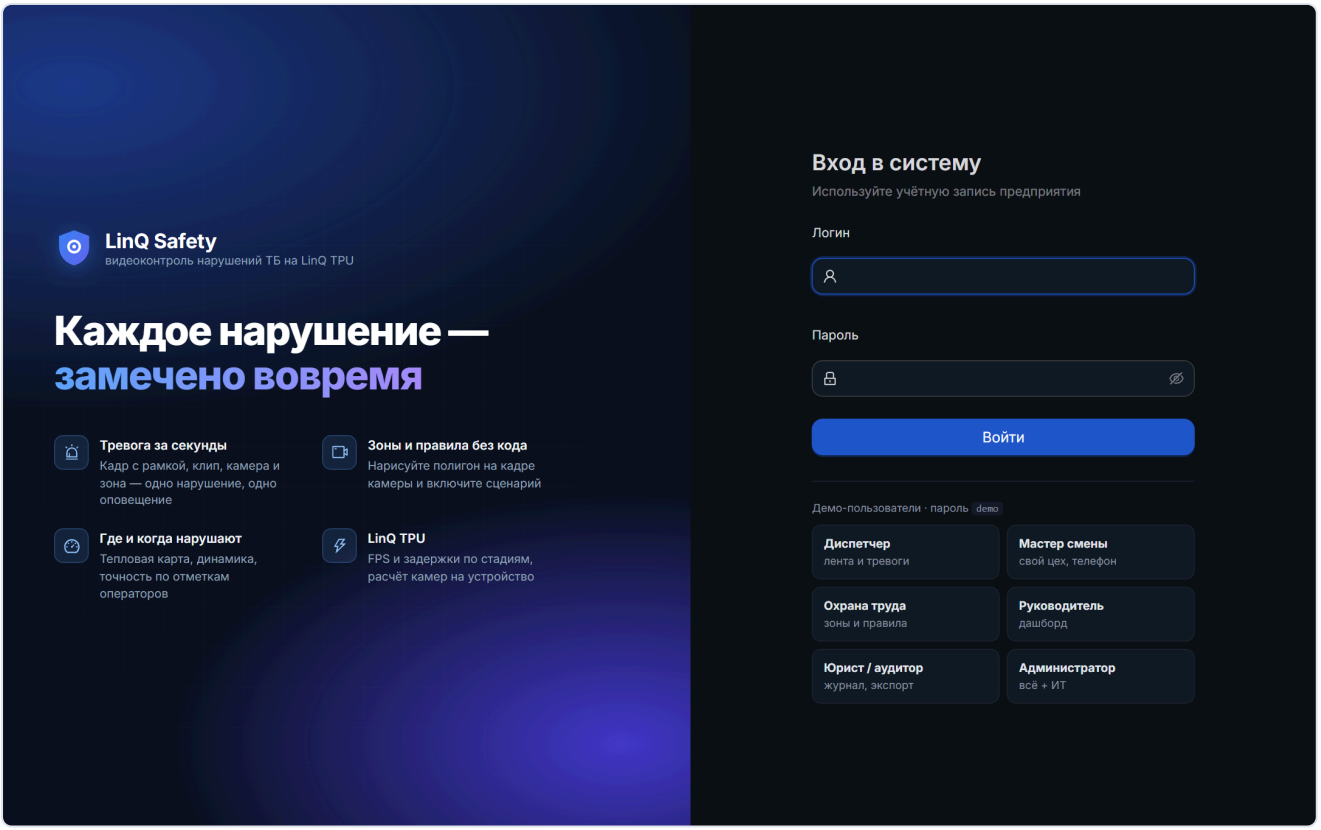
8.6. Вход и токены

- `auth` подписывает токены RS256; ключ 2048 бит генерируется при первом старте и хранится в томе, поэтому токены переживают рестарт.
- Access-токен живёт 15 минут, содержит роль, участки и имя; refresh — 7 дней, одноразовый, в базе хранится только его хэш. Выход отзывает refresh-токен.
- Остальные сервисы проверяют подпись сами по публичному ключу из `/.well-known/jwks.json` (кэш 5 минут) — без запроса к `auth` на каждое действие.
- Токен принимается из заголовка `Authorization: Bearer …` или из параметра `?access_token=` — для `<img>`, `<video>` и WebSocket, где заголовок не передать.

9. Руководство пользователя: экран за экраном

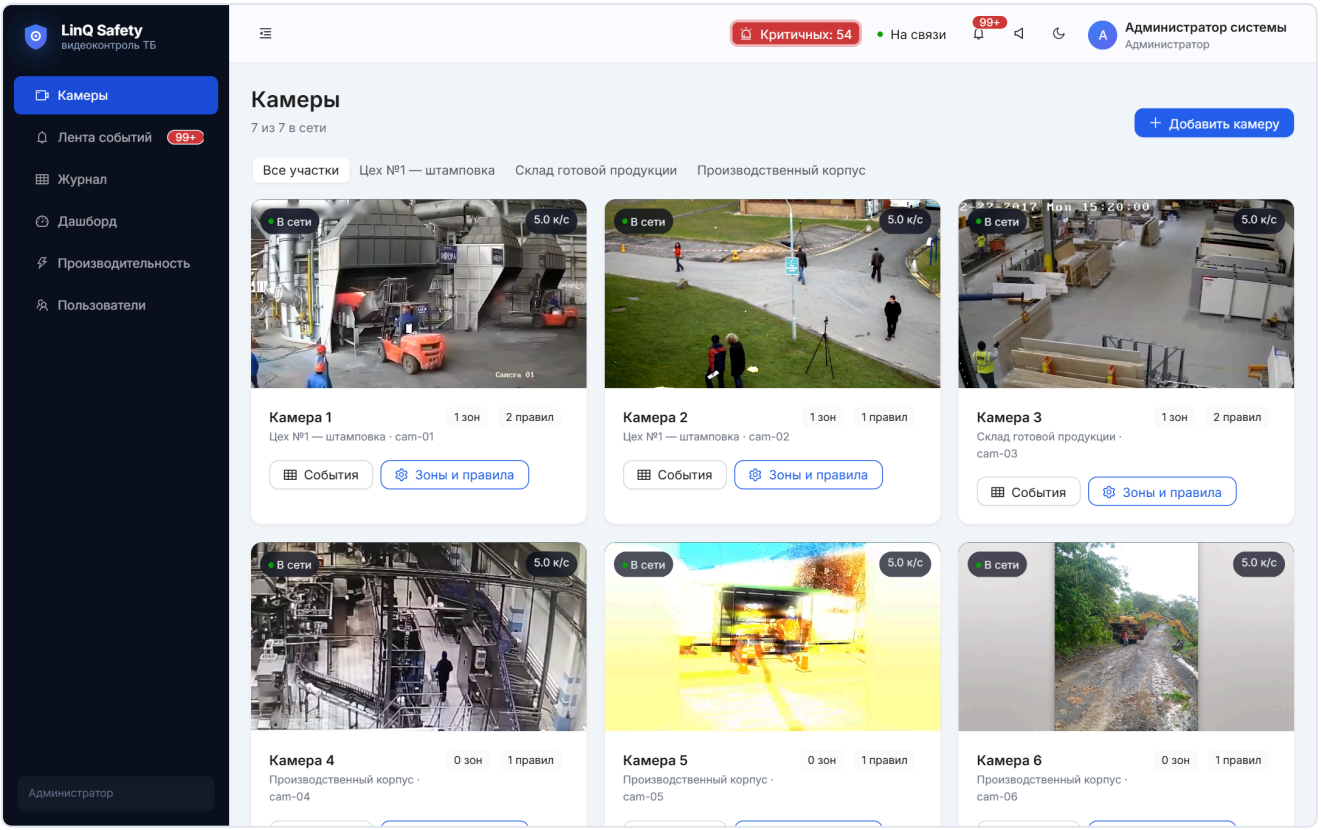
Все скриншоты сняты в работающей системе 24.10.2026: семь камер, инференс на LinQ TPU через брокер. Пересобираются командой `python docs/make_screenshots.py`.

9.1. Вход



Откройте http://localhost:8080. Справа — форма входа и кнопки демо-ролей: один клик входит под нужной ролью с паролем `demo`. После входа каждая роль попадает на свой стартовый экран (§8.5).

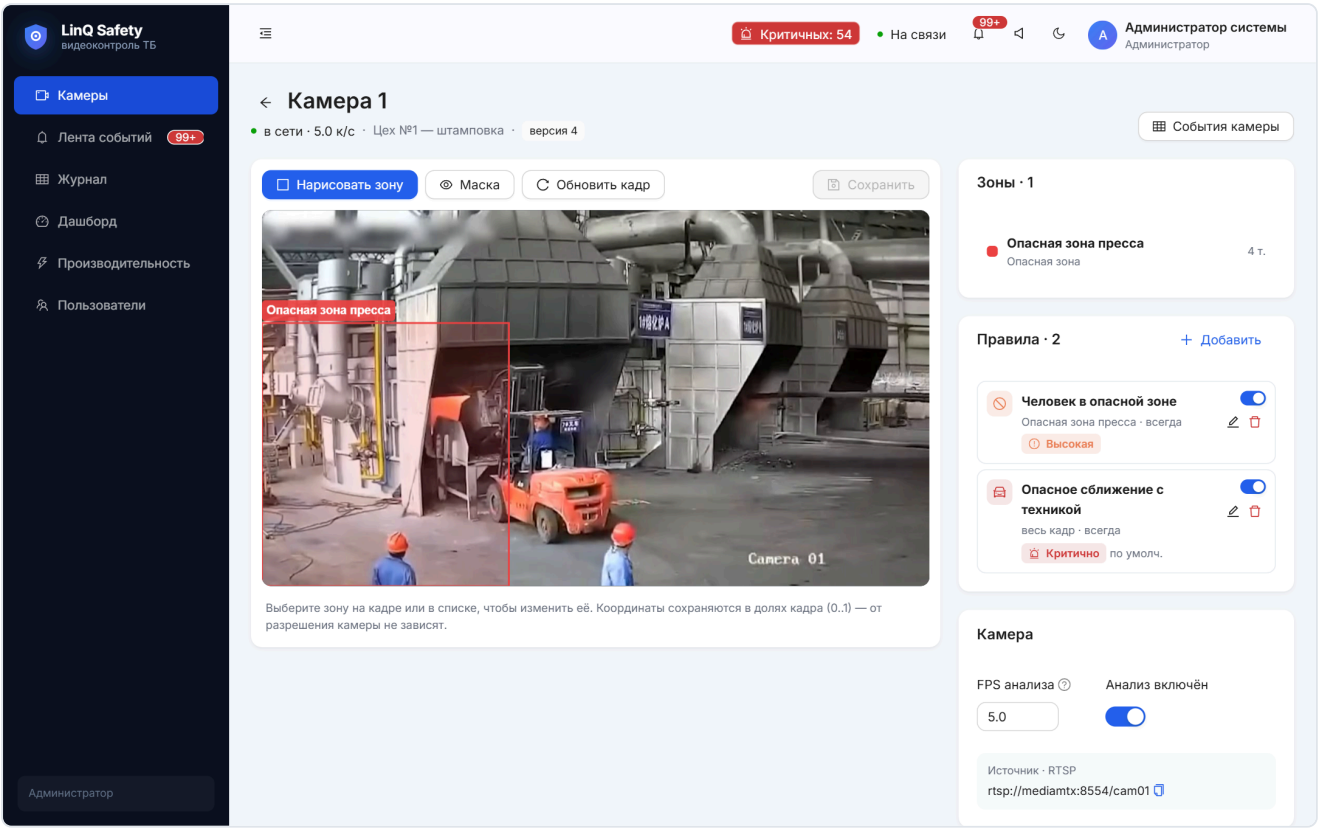
9.2. Камеры



Живое видео всех камер (HLS из MediaMTX) со статусом потока «В сети / Подключение / Нет сигнала» и фактическим FPS анализа. Вкладки сверху фильтруют по участкам. На карточке — число зон и правил, кнопки «События» (журнал этой камеры) и «Зоны и правила». Кнопка «Добавить камеру» открывает форму: название, участок, источник (RTSP, видеофайл или папка кадров), путь и FPS анализа. `ingest` начинает читать новую камеру без перезапуска.

Шапка видна на всех экранах: красная плашка с числом критичных непринятых событий, индикатор связи WebSocket, колокольчик с непринятыми, звук тревог, тёмная тема, профиль.

9.3. Настройка камеры: зоны и правила

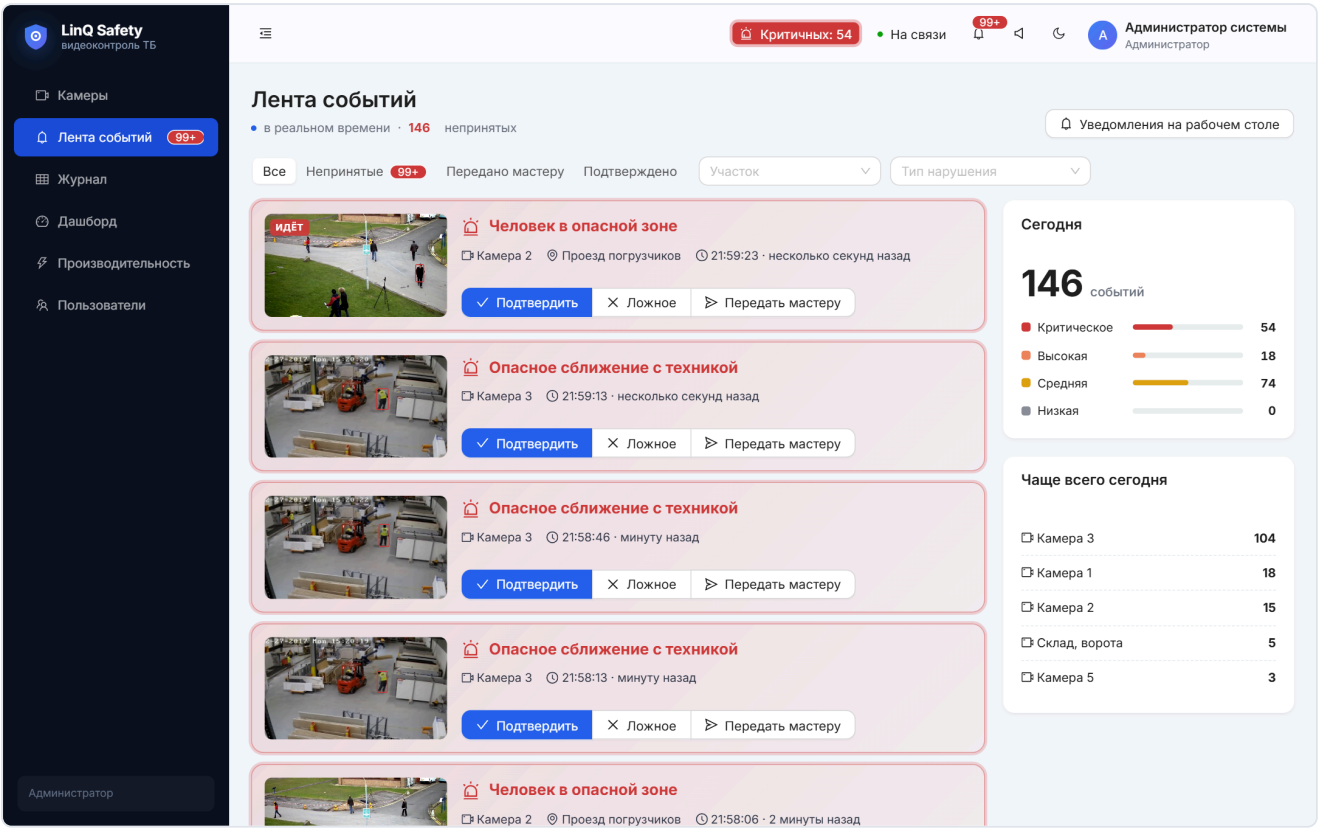


Экран специалиста по охране труда.

1. **Зона.** «Нарисовать зону» — клики по последнему кадру камеры ставят вершины, «Замкнуть» завершает полигон. Вершины и всю зону можно перетаскивать, клик по полупрозрачной точке на ребре добавляет вершину. «Маска» затемняет всё, кроме зон. Координаты хранятся в долях кадра — смена разрешения камеры зоны не сдвигает.
2. **Правило.** «Добавить» в блоке «Правила»: сценарий из каталога, зона или «весь кадр», важность, параметры (форма строится по JSON Schema сценария), расписание (дни недели, интервал) и срок действия. Переключатель на карточке правила включает и выключает его.
3. **Камера.** FPS анализа, «Анализ включён», источник.

Метка «версия N» в заголовке растёт при каждом изменении: по ней видно, что горячий путь перечитал настройки. Изменение применяется к следующему кадру.

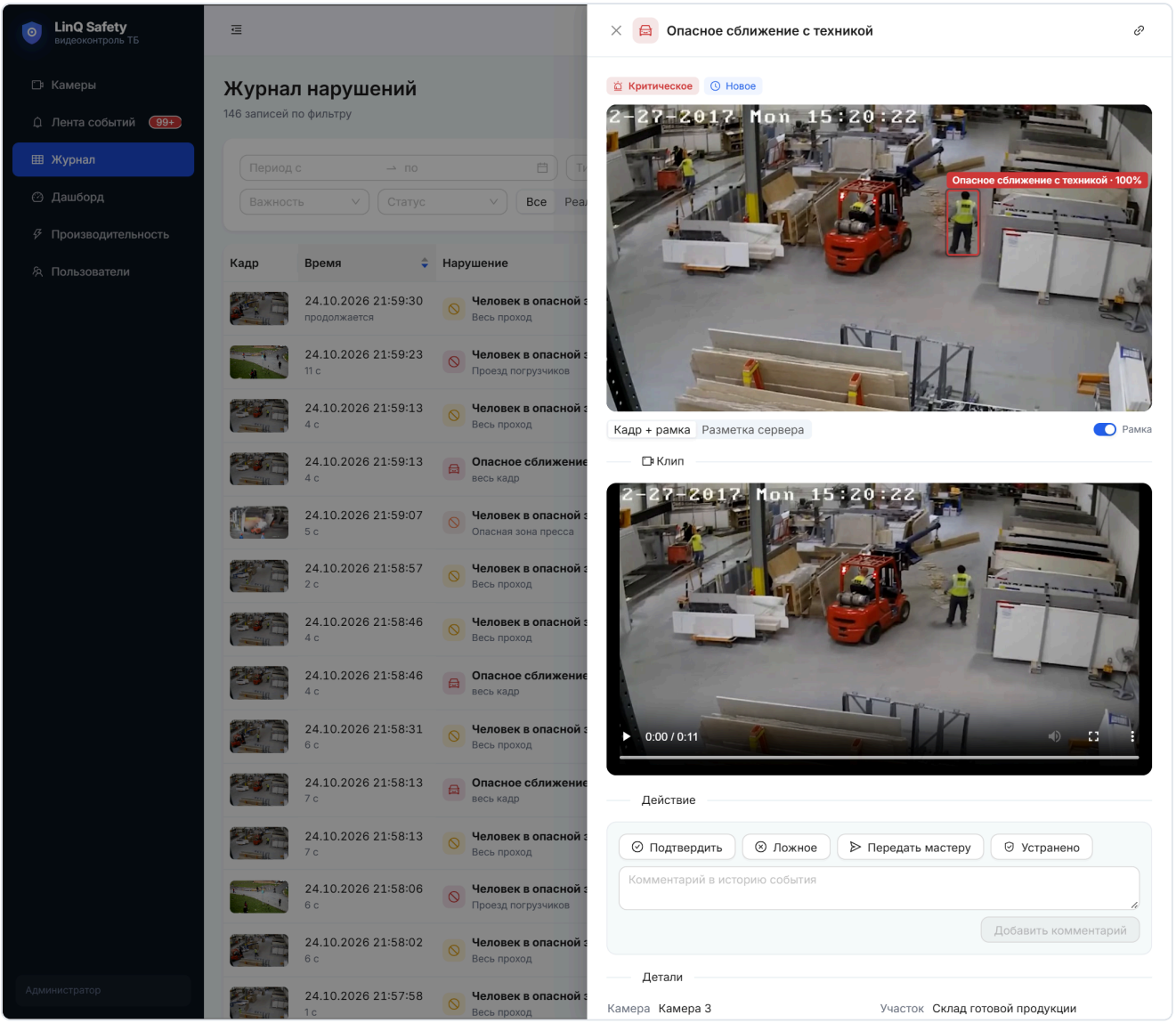
9.4. Лента событий



Рабочий экран диспетчера и мастера. События приходят по WebSocket в реальном времени: кадр с рамкой, нарушение, камера, зона, время. Критичные выделены красным и закреплены, пока их не примут. Кнопки прямо на карточке: «Подтвердить», «Ложное», «Передать мастеру». Вкладки — «Все», «Непринятые», «Переданно мастеру», «Подтверждено»; фильтры по участку и типу нарушения. Справа — счётчики дня по важности и камеры, где сегодня нарушают чаще.

Непринятые события собираются в **одну сводку** в углу экрана (число, сколько из них критичных и не принятых вовремя, последнее событие), а не в стопку всплывающих плашек. Кнопка «Уведомления на рабочем столе» включает системные уведомления: критичные события и эскалации придут, даже если вкладка свёрнута.

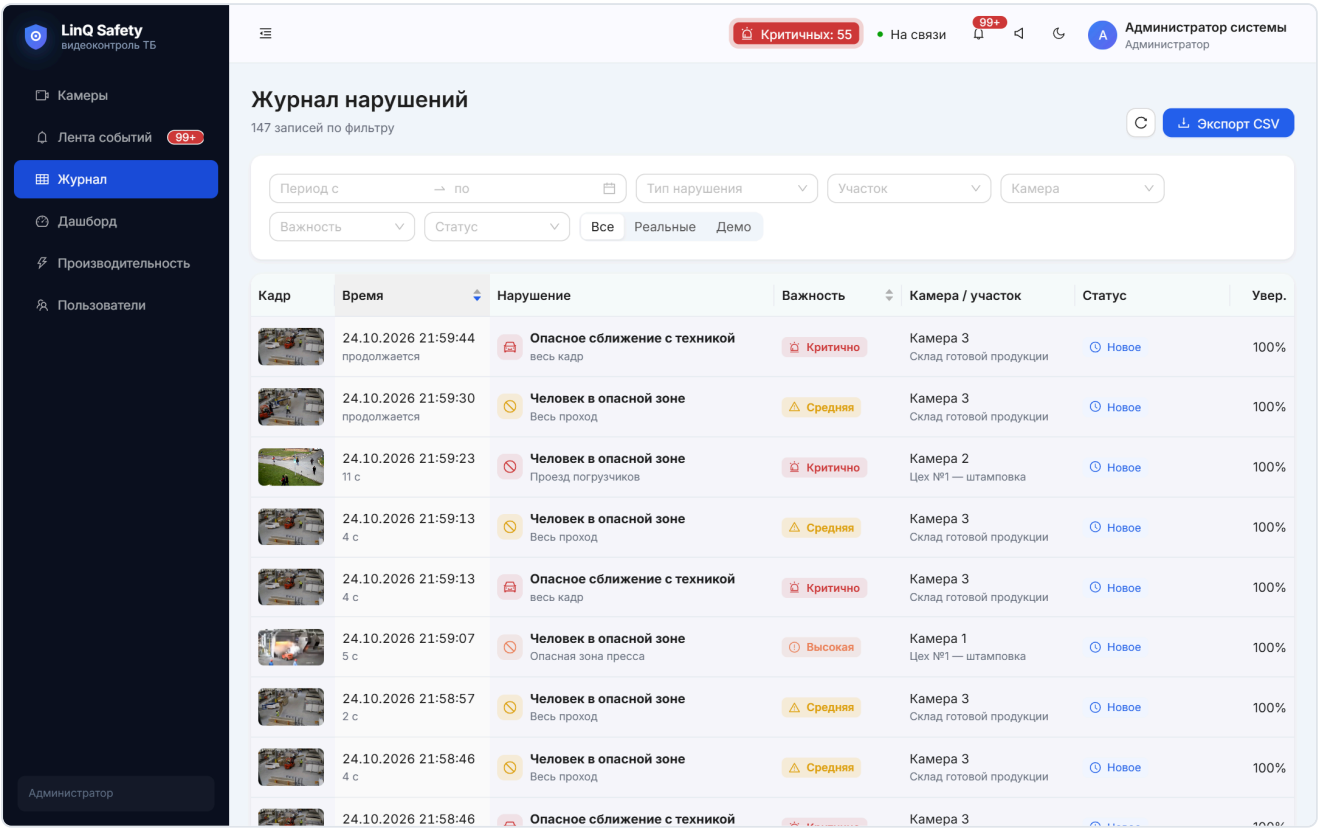
9.5. Карточка события



Открывается кликом по событию на любом экране; ссылкой `?event=<id>` можно поделиться. Внутри:

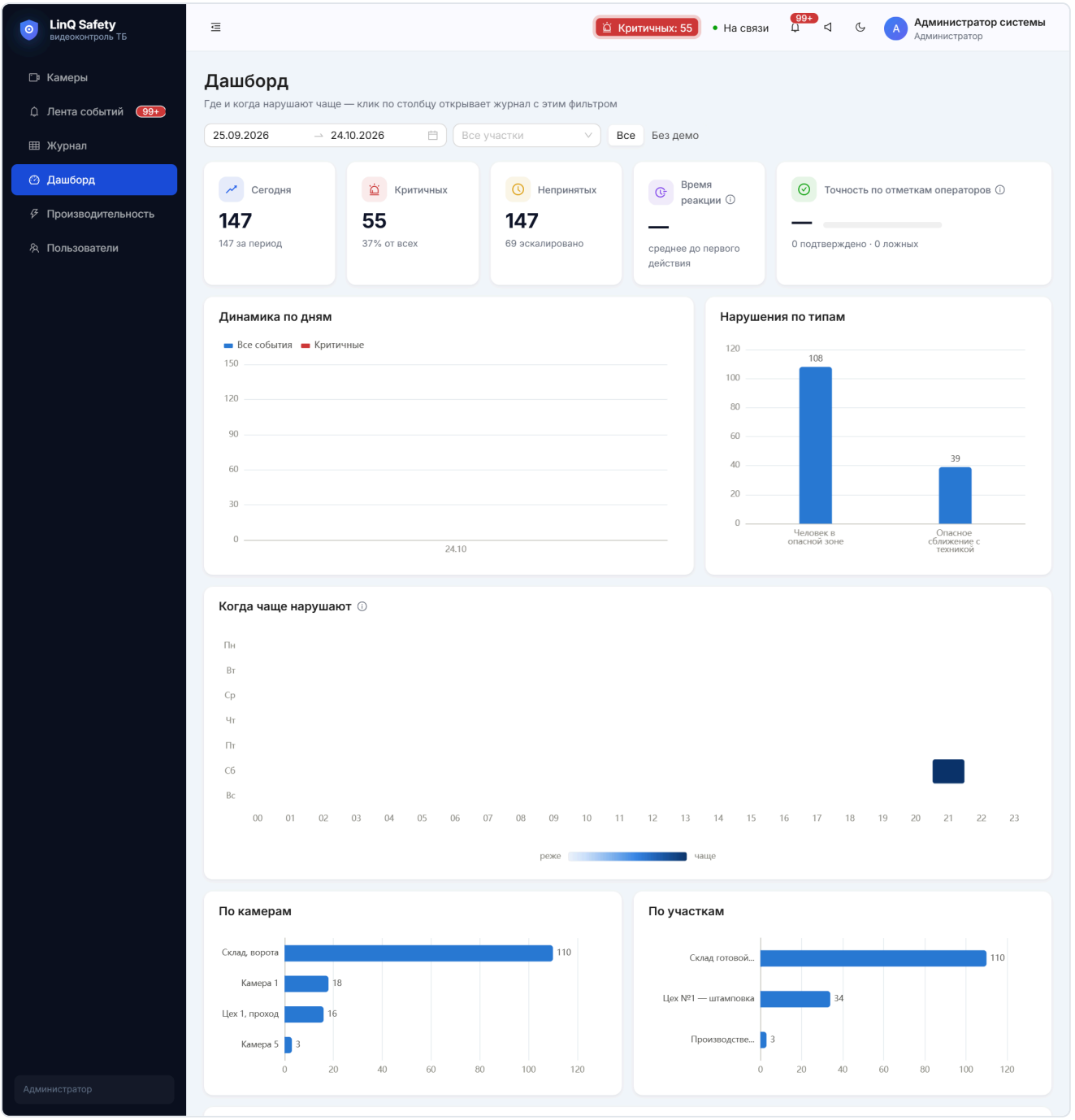
- кадр, подтвердивший нарушение, с рамкой человека (рамку можно скрыть) и вкладка «Разметка сервера» — кадр, размеченный на сервере вместе с зоной;
- клип ±5 с — плеер и кнопка «Скачать»; пока клип собирается, карточка обновится сама;
- действия: «Подтвердить», «Ложное», «Передать мастеру», «Устранено» и комментарий в историю;
- детали: камера, участок, зона, начало, окончание и длительность, уверенность, номер трека;
- история: кто, что и когда сделал, включая системные «создано», «закончилось», «не принято вовремя».

9.6. Журнал



История всех нарушений и действий. Фильтры — период (быстрые: сегодня, вчера, 7 и 30 дней, этот месяц), камера, участок, тип нарушения, важность, статус, «реальные / демо»; фильтры живут в адресе страницы, ссылку можно отправить. Серверная пагинация и сортировка по времени и важности. «Экспорт CSV» выгружает ровно то, что отфильтровано: разделитель «;», UTF-8 с BOM (открывается в Excel), время по Москве, ссылки на кадр и клип, SHA-256 кадра.

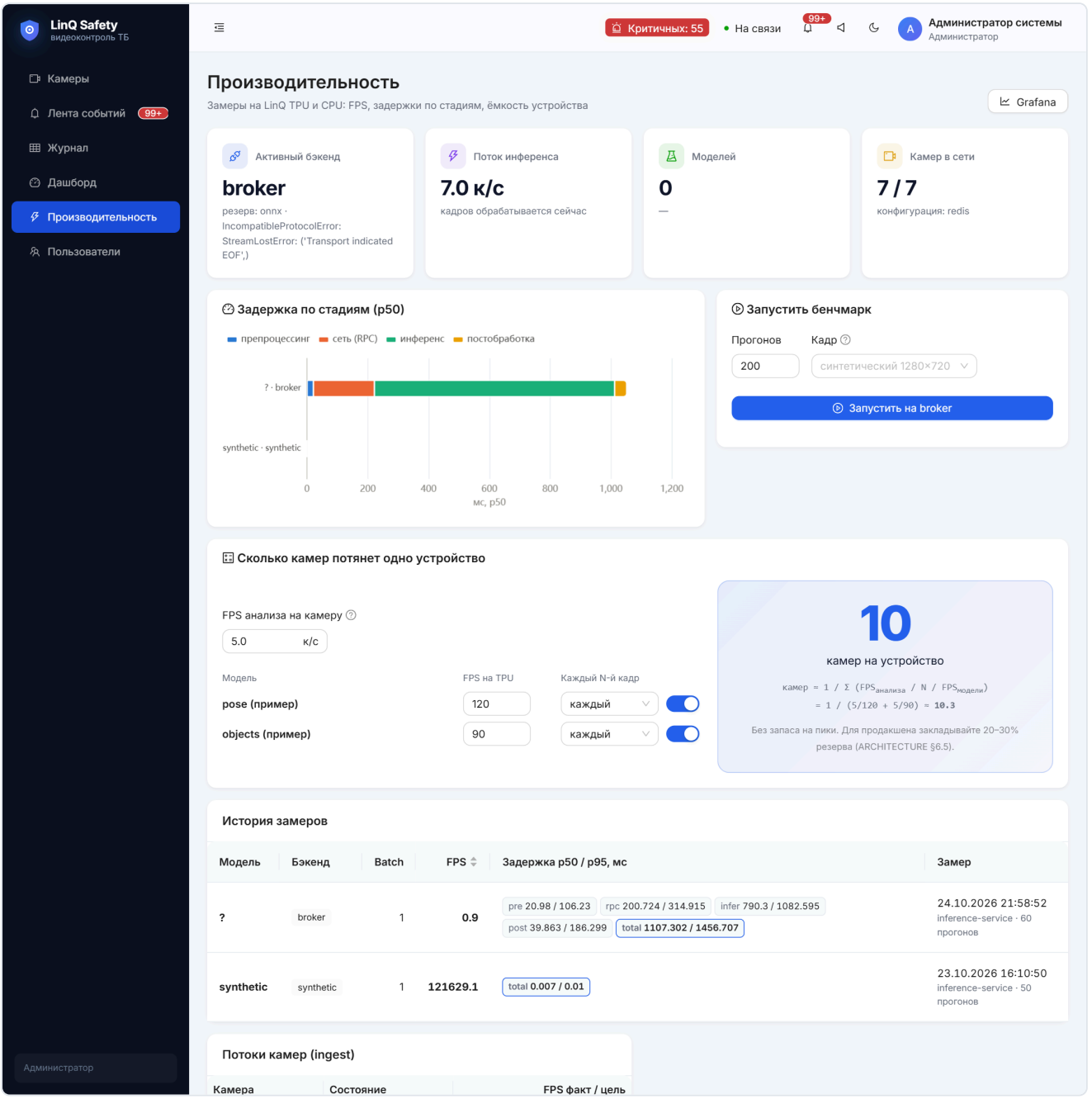
9.7. Дашборд



Экран руководителя и охраны труда: «где и когда нарушают чаще». Плитки — события за сегодня, критичные, непринятые и эскалированные, среднее время до первой реакции, точность по отметкам операторов. Графики — динамика по дням, нарушения по типам, тепловая карта «день недели × час», по камерам и по участкам. Клик по столбцу открывает журнал с этим фильтром. Переключатель «Все / Без демо» отделяет реальные события от демо-истории.

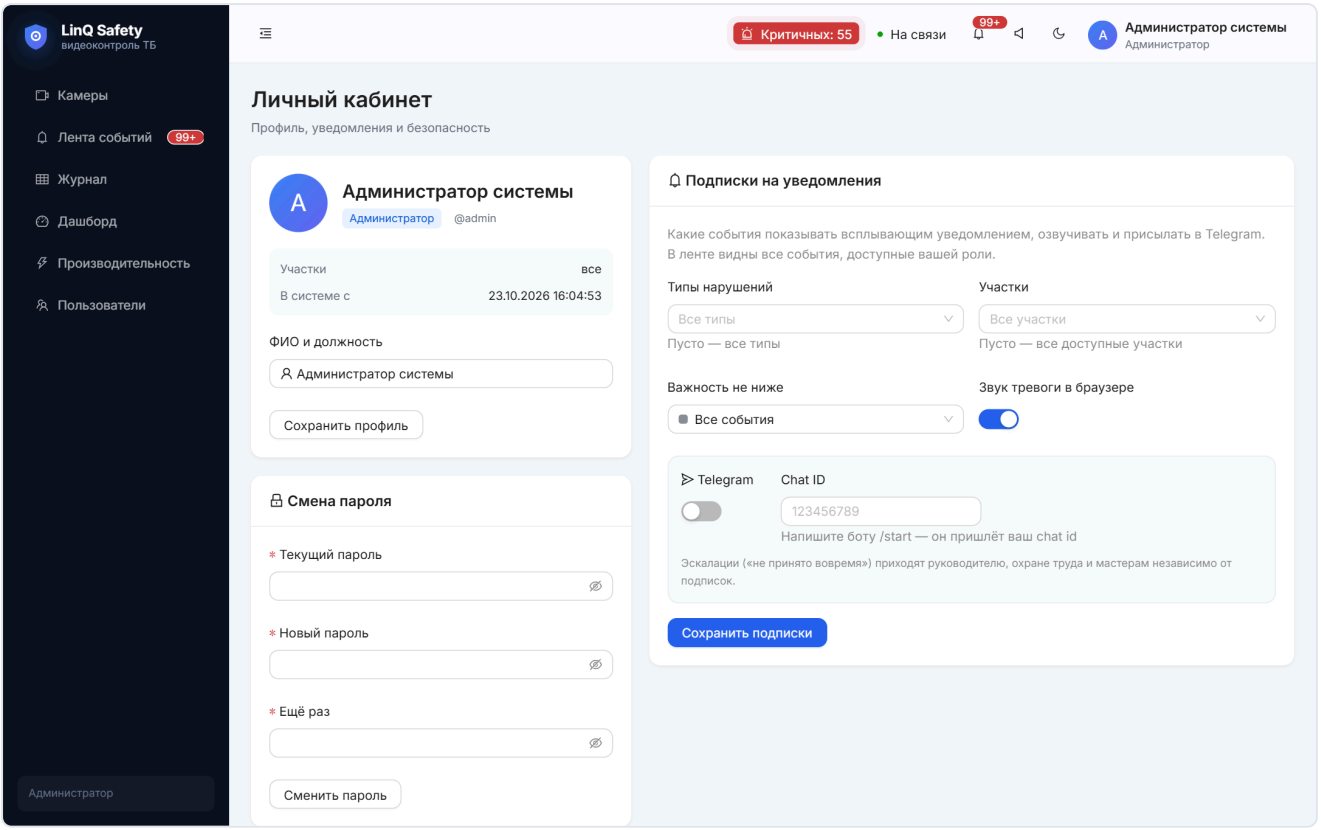
На скриншоте — реальные события одного вечера, поэтому динамика и тепловая карта заполнены одной ячейкой. Для показа истории есть генератор `POST /api/events/demo/seed` (§11.4): такие события помечены `demo=true` и честно отделяются фильтром.

9.8. Производительность



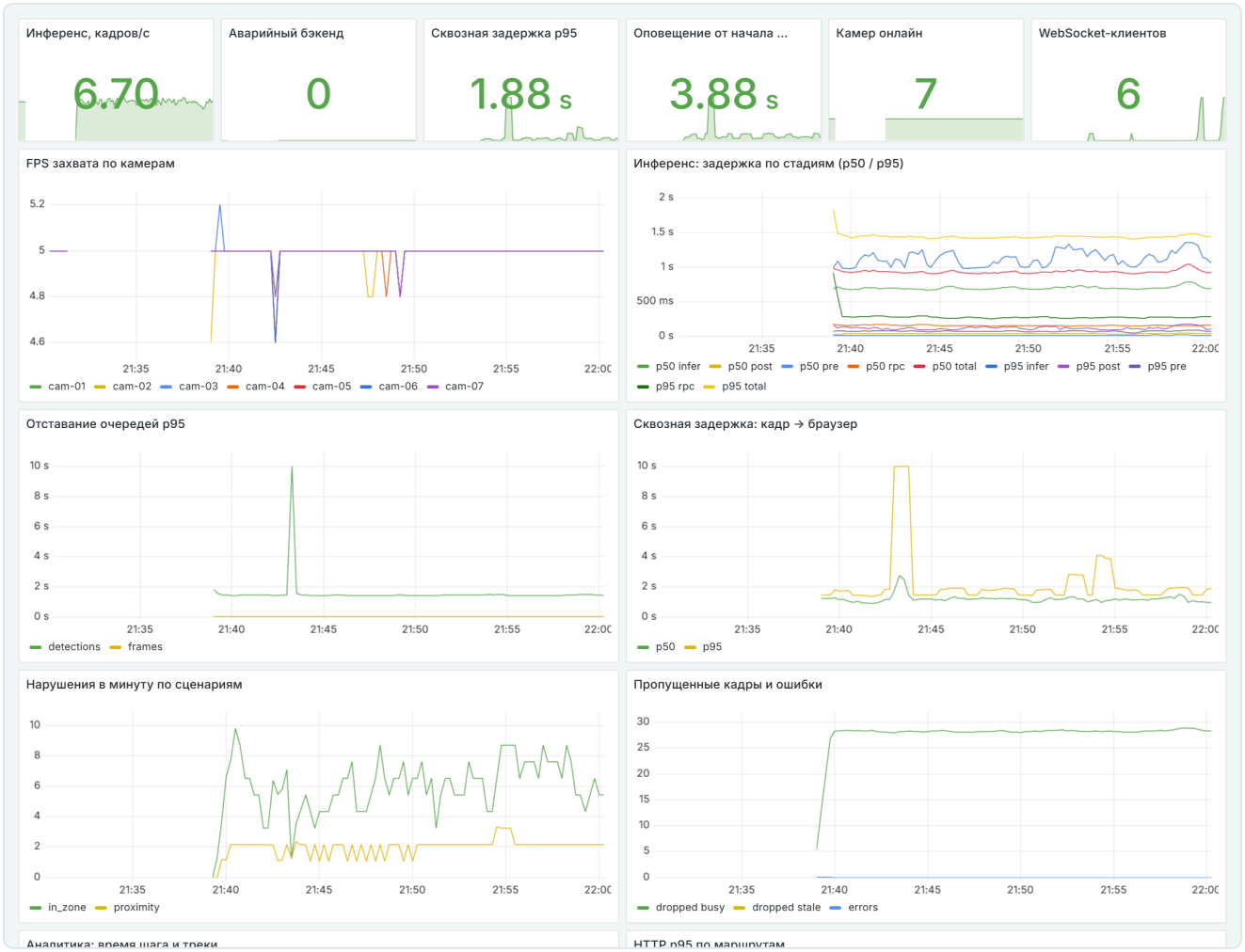
- **Плитки:** активный бэкенд и запасной (с последней ошибкой основного), поток инференса в кадрах/с, камеры в сети.
- **Задержка по стадиям (p50):** препроцессинг, сеть, инференс, постобработка — по каждому замеру.
- **Запустить бенчмарк:** N прогонов одного кадра (синтетического или последнего кадра выбранной камеры) через активный бэкенд; результат — FPS и p50/p95 по стадиям, сохраняется в историю.
- **Сколько камер потянет одно устройство:** калькулятор `камер ≈ 1 / Σ (FPS анализа / N / FPS модели)`, где N — «каждый N-й кадр» для модели. В полях по умолчанию — **примерные** значения FPS моделей (120 и 90), их нужно заменить замером.
- **История замеров** и **потоки камер** (`ingest`): состояние, фактический и целевой FPS.

9.9. Личный кабинет



ФИО и должность; подписки на уведомления — типы нарушений, участки, минимальная важность, звук в браузере, Telegram (chat id присылает бот в ответ на `/start`); смена пароля. Подписки сразу попадают в `notifications` через Redis. Экран «Пользователи» (администратор) — список, создание, роль, участки, отключение.

9.10. Grafana



http://localhost:3000, дашборд «LinQ: конвейер видеоаналитики» открывается без входа. Панели: инференс в кадрах/с, аварийный бэкенд, сквозная задержка p95, оповещение от начала нарушения p95, камеры онлайн, клиенты WebSocket, FPS захвата по камерам, задержка инференса по стадиям, отставание очередей, нарушения в минуту по сценариям, пропущенные кадры и ошибки, время шага аналитики, HTTP p95 по маршрутам.

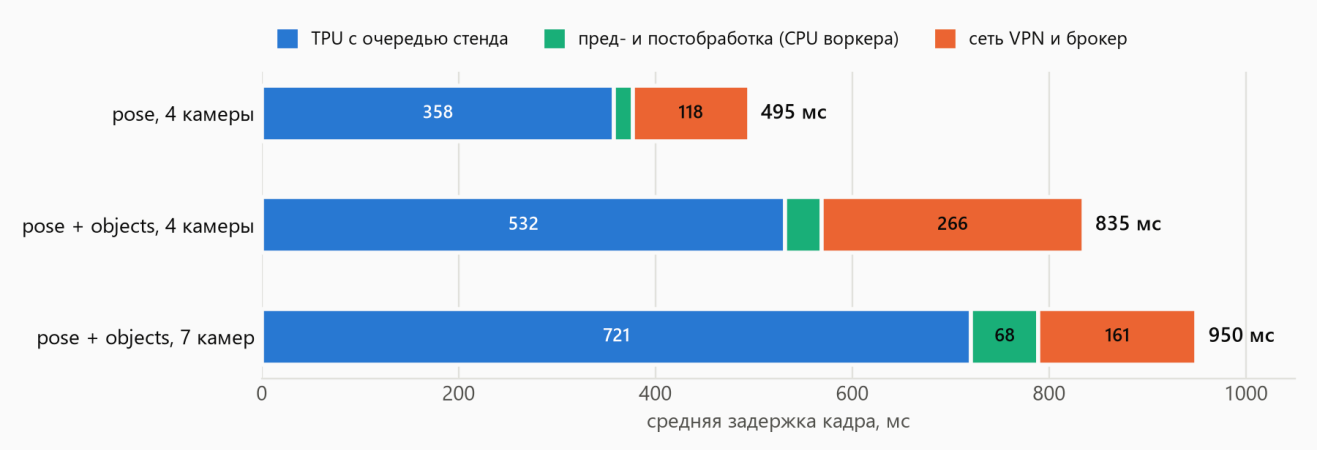
10. Производительность и проверка качества

Всё в этом разделе измерено на работающей системе, и рядом указано, как повторить замер. Там, где величина не измерена, так и написано.

10.1. Как мерили

- **Конвейер:** 7 камер (`data/demo_videos/cam01…cam07.mp4` , 720p) по 5 кадров/с в RTSP, `INFERENCE_BACKEND=broker` , `INFERENCE_CONCURRENCY=7` , 4 воркера на стенде, обе модели на каждом кадре. Числа — из Prometheus за окно 10 минут (задержки по стадиям) и час (сквозные), 24.10.2026, 21:45–22:00 МСК.
- **Ранние замеры** 24.10.2026 днём: 4 камеры, сначала только `pose` , затем `pose + objects` (`backend/ml/worker/README.md`).
- **Где стоит секундомер.** `pre` , `infer` , `post` меряет воркер на стенде (`infer` — от отправки задач на TPU до получения обоих ответов, **включая очередь общего TPU**); `rpc` — всё остальное время запроса: VPN, брокер, ожидание свободного воркера; `queue` — от захвата кадра до взятия в работу.
- **Машина сервера:** Windows 11, Docker Desktop (WSL 2), 16 логических ядер, 15 ГБ памяти для Docker. GPU сервису не нужен.

10.2. Задержка кадра на TPU



Конфигурация	Кадров/с через TPU	Задержка кадра, средняя	TPU с очередью	Пред- и постобработка	Сеть и брокер
`pose` , 4 камеры	6,6	495 мс	358 мс	~20 мс	118 мс
`pose + objects` , 4 камеры	4,3	835 мс	532 мс	37 мс	266 мс
`pose + objects` , 7 камер	6,6	950 мс (p50 928, p95 1 434)	721 мс	68 мс	161 мс
бенчмарк, один поток под нагрузкой 7 камер	0,9	1 114 мс (p50 1 107, p95 1 457)	805 мс	103 мс	207 мс

Повторить: страница «Производительность» → «Запустить бенчмарк», или `POST /api/inference/benchmark {"runs": 60, "camera_id": "cam-03"}` ; поток — Grafana, панель «Инференс: задержка по стадиям».

Главная статья — TPU вместе с очередью **общего** стенда: её мы не контролируем, и она растёт с нагрузкой от всех команд. Пред- и постобработка на CPU воркера — десятки миллисекунд, трекинг и сценарии — меньше 5 мс на кадр (p95).

10.3. От нарушения до оповещения

Что	Значение	Метрика / источник
От кадра, подтвердившего нарушение, до браузера, p50	**1,05 с**	`event_e2e_seconds`, 117 событий
То же, p95	1,90 с	`event_e2e_seconds`
От начала нарушения до оповещения (с окном подтверждения 1–2 с), p50	**2,7 с**	`alert_delay_seconds`
Сквозная проверка `smoke.py`, кадр → браузер	1,15 с	вывод `smoke.py` (§10.9)
Сближение с погрузчиком на `cam-03`, от начала эпизода до тревоги (pose + objects, 4 камеры)	1,6 с	ручной замер по ролику
Человек в зоне, кадр → браузер (только pose, 4 камеры)	0,58 с	ручной замер
Отставание очереди кадров, p50	3 мс	`stream_lag_seconds{stream="frames"}`
Шаг аналитики (трекинг + сценарии), p95	4,8 мс	`analytics_step_seconds`

Цель кейса — оповещение за единицы секунд. Итоговая задержка ≈ окно подтверждения (задаётся правилом осознанно: нарушение должно длиться, а не мелькнуть) + около секунды на TPU, сеть и доставку.

10.4. Бюджет задержки: замысел и факт

Этап	Заложено в архитектуре	Измерено
Захват, декод, JPEG	≤ 100 мс	3,2 мс (p50), 4,9 мс (p95) — `ingest_decode_seconds`
Очередь до инференса	≤ 50 мс	3 мс (p50)
Сеть до TPU через VPN	20–80 мс	161 мс (RTT VPN ~50 мс, плюс брокер и ожидание воркера)
Пред-, постобработка и TPU	«измерить»	790 мс, из них TPU с очередью стенда 721
Трекинг и правила	≤ 10 мс	4,8 мс (p95)
Окно подтверждения	1–3 с	1 с (сближение), 2 с (зона), 3 с (телефон)
Запись в журнал и доставка	≤ 100 мс	входит в 1,05 с кадр → браузер

10.5. Пропускная способность и масштабирование

7 камер по 5 кадров/с дают 35 кадров/с на входе; через TPU стенда проходит 6,6 кадров/с с обеими моделями. Остальные кадры **сознательно пропускаются** (`inference_dropped_total{reason="busy"}` ≈ 28 кадров/с): на TPU всегда идёт самый свежий кадр каждой камеры, очередь не растёт, задержка не накапливается. Фактическая частота анализа — около 1 кадра/с на камеру при семи камерах.

Камер на этом стенде	Анализ на камеру (6,6 кадров/с с двумя моделями)	Подходит для
1–2	3–5 кадров/с	все сценарии с запасом
3	~2 кадра/с	окно подтверждения 1 с — 2 кадра
7	~1 кадр/с	работает, но окно 1 с опирается на 1–2 кадра; лучше окна 2–3 с

Оценка для выделенного устройства — по формуле калькулятора на странице «Производительность»: `камер ≈ FPS_TPU / (FPS анализа на камеру × моделей на кадр)`. Чистый FPS выделенного TPU без очереди стенда мы **не измеряли**: на хакатоне доступен только общий стенд, поэтому числа выше — нижняя граница.

Потребление ресурсов сервером при 7 камерах (`docker stats`, 100 % = одно ядро):

Контейнер	CPU	Память
`ingest` (декод 7 потоков 720p, JPEG, буферы клипов)	50 %	1,0 ГБ
`inference`	7 %	170 МБ
`analytics`	1 %	140 МБ
`minio` / `redis` / `mediamtx`	3 % / 3 % / 5 %	550 / 260 / 100 МБ
остальные сервисы	< 1 % каждый	20–100 МБ
всего	**~0,7 ядра**	**~3 ГБ**

10.6. Качество распознавания

Сводка из §6.5:

Что	Результат
`objects`, mAP50 на отложенном складе	погрузчик 0,36 · рохля 0,28 · телефон 0,53
TPU против CPU на живом конвейере	рамки совпадают, уверенности ±0,05
Корреляция логитов классов TPU и float (`objects`)	0,65 → 0,98 после калибровки с отсечением
Тело + хвост против полной модели	число детекций совпадает, расхождение выходов ≤ 1e-4
Постпроцессинг против Ultralytics	совпадает до рамки: 35 кадров, 204 детекции
`pose` на дальних людях	находит 2–3 из 7 на пешеходном ролике OpenCV — ограничение модели

10.7. Чего мы не измерили

- **Событийные precision и recall, ложные тревоги в час, долю дублей** — на размеченных роликах с производства. Таких роликов с разметкой интервалов нарушений у нас нет; папка `backend/ml/eval/` подготовлена под эту оценку.
- **Чистый FPS TPU** без общей очереди стенда.
- Вместо этого система собирает **онлайн-оценку точности**: отметки операторов «подтверждено / ложное» дают precision по каждому сценарию и камере (`GET /api/events/stats/precision`, плитка на

дашборде), а ложные кадры остаются в MinIO — это готовый набор для дообучения.

10.8. Автотесты

31 тест, `python -m pytest backend/server/tests` — около 2 секунд, без Docker (Redis подменяется `fakeredis`).

Файл	Тестов	Что проверяет
`test_fsm.py`	8	случайный кадр не срабатывает; подтверждение даёт ровно одно событие и конец; мерцание ниже доли не подтверждается; шум внутри активного нарушения не рвёт и не дублирует его; пауза подавляет повтор; после паузы — новое событие; пропавший трек закрывается через `merge_s`; нулевое окно срабатывает сразу
`test_engine.py`	10	человек в зоне — одно событие, вне зоны — ни одного; перекрытие и новый номер трека склеиваются; два человека — два события; удалённое правило закрывает нарушение; кадры не по порядку игнорируются; ByteTrack работает; YAML и каталог сценариев; рестарт не задваивает активное нарушение и закрывает его, если человек ушёл
`test_object_scenarios.py`	7	сближение рядом и далеко от погрузчика; водитель и слабая детекция не считаются; память о технике `memory_s`; телефон в руке, вдали от рук и головы, без позы
`test_common.py`	6	расписание (окно, через полночь, срок действия, псевдонимы полей); стабильность шардов; шина: подтверждение и DLQ, Pub/Sub

Проверки ML-пайплайна — отдельными скриптами на реальных кадрах: `verify_split.py` (разрез графа), `smoke_local.py` (воркер без TPU), `smoke_broker.py` (обмен с RabbitMQ: JPEG/JSON, битый кадр, таймаут, переход на ONNX).

10.9. Сквозная проверка

`python backend/server/tools/smoke.py` проходит путь пользователя через шлюз. Вывод на работающей системе 24.10.2026:

```
[OK]   вход как operator (Диспетчер)
[OK]   камеры: cam-01=online/5.0fps, cam-02=online/5.0fps, cam-03=online/5.0fps, cam-04=online/5.0fps, cam-05=online/5.0fps,
…
[OK]   сценарии: in_zone, phone, proximity
[OK]   инференс: broker, 6.7 кадр/с
[OK]   WebSocket: hello operator
[OK]   событие in_zone на cam-03, снапшот /media/linq/events/2026/10/24/525f6fd2-….jpg, задержка кадр→браузер 1.15 с
[OK]   снапшот доступен: 200, 113378 байт
[OK]   статус → confirmed, действий в истории: 2
[OK]   журнал: всего 165
[OK]   статистика: сегодня 165, непринятых 164, precision 1.0
[OK]   клип: ready /media/linq/clips/2026/10/24/525f6fd2-….mp4
Готово.
```

11. API

Каждый сервис описан в Swagger: `http://localhost:8080/api/<auth|config|events|inference|ingest|analytics|notifications>/docs`. Все запросы, кроме входа и служебных, требуют `Authorization: Bearer <access_token>`. У каждого сервиса есть `GET /health` и `GET /metrics`.

11.1. `auth` — `/api/auth`

Метод и адрес	Что делает	Кто
`POST /login`	вход по логину и паролю → access, refresh, профиль	все
`POST /refresh`	новая пара токенов (refresh одноразовый)	все
`POST /logout`	отозвать refresh-токен	все
`GET /.well-known/jwks.json`	публичный ключ для проверки JWT	сервисы
`GET /roles`	справочник ролей	все
`GET /me` · `PATCH /me`	профиль, подписки на уведомления, Telegram	вошедший
`POST /me/password`	сменить пароль	вошедший
`GET /users` · `POST /users` · `PATCH /users/{id}`	пользователи, роли, участки, отключение	администратор

11.2. `camera` — `/api/config`

Метод и адрес	Что делает
`GET /tree`	дерево «площадка → участки → камеры» для меню
CRUD `/sites`, `/areas`, `/cameras`	площадки, участки, камеры (`source_type`: `rtsp` | `file` | `images`)
`GET /cameras/{id}`	камера с зонами, правилами и статусом потока
`GET /cameras/{id}/config`	конфигурация камеры так, как её видит горячий путь
`GET /cameras/{id}/snapshot`	последний кадр камеры (JPEG) — фон редактора зон
CRUD `/cameras/{id}/zones`	зоны-полигоны (удаление зоны удаляет её правила)
CRUD `/cameras/{id}/rules`	правила: сценарий, зона, параметры (проверяются по JSON Schema), важность, расписание, срок
`GET /scenarios`	каталог сценариев с JSON Schema параметров

Чтение — любой вошедший; изменения — охрана труда и администратор. После каждого изменения конфигурация камеры уходит в Redis.

11.3. `events` — `/api/events`

Метод и адрес	Что делает
`GET /events`	журнал: фильтры, пагинация (`page` , `page_size` ≤ 500), сортировка `-started_at` | `started_at` | `-severity` | `-created_at`
`GET /events/{id}`	карточка с историей действий и допустимыми переходами `allowed_statuses`
`PATCH /events/{id}/status`	сменить статус с комментарием и назначением
`POST /events/{id}/comments`	комментарий в историю
`GET /events/export.csv`	экспорт по тем же фильтрам (до 50 000 строк)
`GET /stats/summary`	KPI: всего, сегодня, критичных, непринятых, эскалированных, время реакции, precision
`GET /stats/by-type` · `/by-camera` · `/by-area`	разрезы
`GET /stats/heatmap`	день недели × час
`GET /stats/trend`	по дням (по умолчанию 30 дней)
`GET /stats/precision?by_camera=true`	доля подтверждённых среди отмеченных — по сценариям и камерам
`POST /ingest/violations`	принять нарушение по HTTP (заголовок `X-Ingest-Token`), опционально с кадром в base64
`POST /demo/seed` · `DELETE /demo`	сгенерировать или удалить демо-историю (`demo=true`), администратор

Фильтры одинаковы для журнала, CSV и статистики; списочные параметры повторяются:

```
GET /api/events/events?camera_id=cam-01&camera_id=cam-02&area_id=1&scenario=proximity&severity=critical
    &status=new&from=2026-10-24T00:00:00%2B03:00&to=2026-10-25T00:00:00%2B03:00&demo=false&page=1&page_size=50&sort=-
started_at
```

11.4. `inference` , `ingest` , `analytics` , `notifications`

Метод и адрес	Что делает
`GET /api/inference/models`	активный бэкенд, модели, аварийный режим, параллельность, FPS
`POST /api/inference/detect`	детекция по одному кадру: тело — JPEG (`Content-Type: image/jpeg`)
`POST /api/inference/detect/upload`	то же файлом (удобно из Swagger)
`POST /api/inference/benchmark`	`{"runs": 200, "warmup": 20, "camera_id": "cam-01"}` → FPS и p50/p95 по стадиям
`GET /api/inference/benchmarks`	история замеров (сервис + JSON-отчёты из `backend/ml/benchmark/out/`)
`GET /api/ingest/streams`	потоки камер: состояние, FPS, ошибки, переподключения, размер буфера
`POST /api/ingest/streams/{cam}/restart`	перезапустить захват камеры

Метод и адрес	Что делает
`GET /api/analytics/scenarios` · `/scenarios/errors`	загруженные сценарии; модули, которые не загрузились
`GET /api/analytics/state`	по камерам: треки, автоматы, активные нарушения, отставание
`WS /ws?access_token=…`	события в реальном времени (§8.3)
`GET /api/notifications/connections`	кто сейчас на связи

11.5. Примеры

```
TOKEN=$(curl -s -X POST localhost:8080/api/auth/login -H 'Content-Type: application/json' \
  -d '{"username":"hse","password":"demo"}' | python -c "import sys,json;print(json.load(sys.stdin)['access_token'])")
H="Authorization: Bearer $TOKEN"

curl -H "$H" "localhost:8080/api/events/events?severity=critical&status=new&page_size=10"
curl -H "$H" "localhost:8080/api/events/stats/summary?demo=false"
curl -H "$H" -o journal.csv "localhost:8080/api/events/events/export.csv?from=2026-10-24T00:00:00%2B03:00"

# включить на камере правило «сближение с техникой» с окном 1 с
curl -H "$H" -H 'Content-Type: application/json' -X POST localhost:8080/api/config/cameras/cam-03/rules \
  -d '{"scenario":"proximity","severity":"critical","params":{"distance_heights":1.0,"confirm_s":1.0}}'

# детекция одного кадра активным бэкендом
curl -X POST localhost:8080/api/inference/detect -H 'Content-Type: image/jpeg' --data-binary @frame.jpg
```

12. Установка и запуск

12.1. Что нужно

- Docker с Compose v2 (проверено на Docker Desktop, Windows 11 / WSL 2), около 4 ГБ свободной памяти.
- Свободные порты: 8080 (шлюз), 3000 (Grafana), 9090 (Prometheus), 9000/9001 (MinIO), 5432, 6379, 8554, 8888, 8889, 8081.
- Для режима TPU — доступ к VPN стенда LinQ (`artifacts/Hakathon.ovpn`, логин и пароль VPN, учётка Jupyter).

12.2. Быстрый запуск

```
cp .env.example .env                # необязательно: без .env работают значения по умолчанию
docker compose up -d --build
python backend/server/tools/smoke.py # сквозная проверка
```

Откройте http://localhost:8080, войдите кнопкой любой демо-роли. По умолчанию `INFERENCE_BACKEND=synthetic`: нейросети и стенд не нужны, камеры без роликов показывают тестовую таблицу.

12.3. Демо-ролики

Ролики в git не лежат (до 180 МБ каждый). Файл `data/demo_videos/<путь RTSP>.mp4` контейнер `cams` крутит по кругу в `rtsp://mediamtx:8554/<путь>`; камеры из списка `DEMO_CAMS` без ролика показывают тестовую таблицу. Все ролики — 720p H.264 без звука, права у авторов, только для локального демо; источники — в README.md.

```
ffmpeg -ss 465 -to 482.5 -i source.mp4 -an -c:v libx264 -pix_fmt yuv420p data/demo_videos/cam01.mp4
```

12.4. Инференс на LinQ TPU

```
# 1. VPN и доступ к Jupyter — в контейнере (нужны secrets/vpn-auth.txt и secrets/jupyter.env)
docker compose --profile linq up -d linq-vpn
docker compose exec -T linq-vpn ljup login
# 2. Воркеры на стенде (бандл и программы уже развёрнуты, §5.6)
docker compose exec -T linq-vpn ljup sh 'sh ~/work/cvhack_worker/backend/ml/worker/start_workers.sh'
# 3. В .env: INFERENCE_BACKEND=broker, INFERENCE_FALLBACK=onnx, INFERENCE_CONCURRENCY=4, DEMO_CAMS="cam01 … cam07"
docker compose up -d
```

Проверка: `GET /api/inference/models` показывает `active_backend: broker`, в детекциях `backend: linq-tpu`, на странице «Производительность» — поток кадров. Если стенд не отвечает, система продолжит работать на CPU, а в плитке бэкенда появится причина.

Развернуть бандл заново: `python3 backend/ml/worker/package.py data/models/pose data/bundles/cvhack_worker.zip`, загрузить через `ljup put`, распаковать в `~/work/cvhack_worker` (подробно — backend/ml/worker/README.md).

12.5. Переменные окружения

Переменная	По умолчанию	Что задаёт
`INFERENCE_BACKEND`	`synthetic`	`synthetic` | `null` | `onnx` | `broker` | `remote` | `pytpu` (§1.5)
`INFERENCE_FALLBACK`	`none`	запасной бэкенд: `onnx` | `synthetic` | `null` | `none`
`INFERENCE_CONCURRENCY`	1	кадров одновременно в детекторе, по одному на камеру; для `broker` — 4 и больше
`INFERENCE_MODELS`	`pose`	модели из `data/models` для `onnx` и аварийного режима (`pose,objects`); в режиме `broker` набор задают воркеры
`MAX_FRAME_AGE_S`	2.0	кадр старше не отправляется в модель
`BROKER_URL` · `BROKER_REQUEST_QUEUE` · `BROKER_TIMEOUT_S`	`amqp://guest:guest@linq-vpn:5672/%2F` · `cvhack12731.jpeg` · 3.0	RabbitMQ стенда и наша очередь
`REMOTE_URL`	—	адрес HTTP-воркера для `remote`
`FRAME_SHARDS`	1	шардов потока кадров
`TRACKER`	`bytetrack`	`bytetrack` | `iou`
`DEMO_CAMS`	`cam01 cam02 cam03`	какие RTSP-пути имитирует контейнер `cams`
`DEMO_PASSWORD`	`demo`	пароль демо-пользователей при первом запуске
`ACCESS_TTL_MIN` · `REFRESH_TTL_DAYS`	15 · 7	срок жизни токенов
`ESCALATE_{CRITICAL,HIGH,MEDIUM,LOW}_MIN`	2 · 5 · 15 · 60	пороги эскалации
`TELEGRAM_BOT_TOKEN`	—	пусто — Telegram выключен
`INGEST_TOKEN`	`change-me`	токен приёма нарушений по HTTP
`GATEWAY_PORT`	8080	порт шлюза на хосте
`POSTGRES_*` , `REDIS_PASSWORD` , `MINIO_ROOT_*` , `MINIO_BUCKET` , `GRAFANA_PASSWORD`	`linq` / `linq-redis` / `minioadmin` / `linq` / `admin`	учётки инфраструктуры — сменить для любого развёртывания вне ноутбука

12.6. Адреса

Что	Адрес
Интерфейс и API	http://localhost:8080
Swagger сервисов	http://localhost:8080/api/{auth,config,events,inference,ingest,analytics,notifications}/docs

Что	Адрес
WebSocket	ws://localhost:8080/ws?access_token=...
Живое видео (HLS)	http://localhost:8080/live/cam01/index.m3u8
Снапшоты и клипы	http://localhost:8080/media/linq/...
Grafana	http://localhost:3000
Prometheus	http://localhost:9090
MinIO	http://localhost:9001 (minioadmin / minioadmin)
Traefik	http://localhost:8081

12.7. Разработка

```
# бэкенд: тесты без Docker
uv venv --python 3.12 .venv
cd backend/server
uv pip install --python ../../.venv -e libs/common -r services/analytics/requirements.txt \
    -r services/inference/requirements.txt -r services/auth/requirements.txt -r services/camera/requirements.txt pytest
fakeredis
../../.venv/Scripts/python -m pytest tests          # Linux/macOS: ../../.venv/bin/python

# фронтенд с горячей перезагрузкой (бэкенд — в compose)
cd frontend && npm install && npm run dev           # http://localhost:5173, прокси на шлюз
npm run dev:mock                                     # без бэкенда, на MSW-моках
```

Любой сервис можно запустить локально против инфраструктуры из compose, задав `DATABASE_URL`, `REDIS_URL`, `MINIO_ENDPOINT` и `AUTH_JWKS_URL` (пример — в README.md).

13. Структура репозитория

```
govnohack/
├── DOCUMENTATION.md                  # этот документ
├── Техническая_документация.pdf      # он же для печати (python docs/make_pdf.py)
├── BRIEF.md, Краткое_описание.pdf    # краткое описание на 5 страниц (make_pdf.py --brief)
├── README.md                         # запуск, сервисы, контракты — коротко
├── docker-compose.yml                # вся система одной командой
├── .env.example                      # переменные окружения с пояснениями
├── artifacts/                        # материалы кейса и стенда LinQ, профиль VPN (в git не входят)
├── backend/
│   ├── server/
│   │   ├── libs/common/common/       # messages.py (контракты), bus.py (Redis Streams), auth.py (JWT),
│   │   │                             # camconfig.py, schedule.py, storage.py, db.py, metrics.py, service.py
│   │   ├── services/
│   │   │   ├── auth/                 # вход, роли, JWT, профиль
│   │   │   ├── camera/               # камеры, зоны, правила
│   │   │   ├── ingest/               # захват, кольцевой буфер, клипы
│   │   │   ├── inference/            # app/, detectors/ (synthetic, onnx, broker, remote, fallback),
│   │   │   │                         # backends/ (onnx), ml_runtime/ (letterbox, хвост, NMS)
│   │   │   ├── analytics/            # app/, engine/ (трекинг, автомат), scenarios/ (in_zone, proximity, phone)
│   │   │   ├── events/               # журнал, статистика, эскалация, демо-история
│   │   │   ├── notifications/        # WebSocket, Telegram
│   │   │   └── Dockerfile            # общий образ сервисов
│   │   ├── infra/                    # traefik, mediamtx, cams, linq-vpn, prometheus, grafana, cameras.yml
│   │   ├── tools/                    # replay.py (ролик → события офлайн), smoke.py (сквозная проверка)
│   │   └── tests/                    # 31 тест
│   └── ml/
│       ├── datasets/build_objects.py # LOCO + COCO → YOLO-набор
│       ├── training/train_objects.py # дообучение YOLOv8s
│       ├── export/                   # export_onnx.py, cut_head.py, verify_split.py
│       ├── compile/                  # Dockerfile с SDK LinQ, compile_tpu.py
│       ├── worker/                   # linq_worker.py, start_workers.sh, package.py, smoke_*.py
│       ├── benchmark/out/            # JSON-отчёты замеров (их показывает GET /api/inference/benchmarks)
│       └── eval/                     # место под событийные метрики
├── data/
│   ├── models/pose/                  # meta.json, yolov8n-pose.onnx, pose_tail.onnx, .tpu      (в git)
│   ├── models/objects/               # meta.json, yolov8s-objects.onnx/.pt, objects_tail.onnx, .tpu (в git)
│   └── demo_videos/ calib/ datasets/ # не в git
├── docs/
│   ├── ARCHITECTURE.md               # замысел и его обоснование (шире реализованного)
│   ├── plans/                        # планы команды по ролям
│   ├── diagrams/                     # схемы 1–7 (SVG, PNG) и генератор build.py
│   ├── img/                          # скриншоты и графики этого документа
│   ├── make_screenshots.py           # скриншоты работающего сервиса
│   ├── make_charts.py                # графики
│   ├── make_pdf.py                   # PDF-версия этого документа
│   └── cdp.py                        # клиент headless Chrome для двух скриптов выше
└── frontend/                         # SPA: React 18, TypeScript, Vite, Ant Design 5
    └── src/                          # api/, store/, components/, pages/, shared/, mocks/
```

14. Технологии и библиотеки

Часть	Что используем	Роль
Язык бэкенда	Python 3.12	все сервисы; тот же язык, что у SDK LinQ
Веб-фреймворк	FastAPI 0.162, Uvicorn 0.54, Pydantic 2.14	REST, WebSocket, контракты сообщений, Swagger
Очереди и кэш	Redis 7 (Streams, Pub/Sub), redis-py 8.1	горячий путь, рассылка, кадры, состояние
База	PostgreSQL 16, SQLAlchemy 2.1 (async), asyncpg 0.32	пользователи, конфигурация, журнал
Объектное хранилище	MinIO, minio-py 7.2	кадры и клипы
Видео	MediaMTX 1.9, FFmpeg, OpenCV 5.0	RTSP-камеры, HLS в браузер, декод, JPEG, клипы H.264
Нейросети	Ultralytics 8.4.0 (YOLOv8), ONNX, ONNX Runtime 1.31 (сервер) / 1.29 (стенд), SDK LinQ `tpu_framework` 2026.9.45	обучение, экспорт, CPU-инференс, квантование и компиляция
Связь со стендом	RabbitMQ (стенд), pika 1.4, OpenVPN, socat	JPEG → TPU → JSON
Аналитика	`trackers` 2.6 / supervision 0.31 (ByteTrack), Shapely 2.2, NumPy 2.5	трекинг, геометрия зон
Безопасность	PyJWT 2.15 (RS256), argon2-cffi 25.1	токены, пароли
Наблюдаемость	prometheus-client 0.26, Prometheus 2.54, Grafana 11.2, structlog 26 (JSON-логи)	метрики, дашборд, логи
Шлюз	Traefik 3.1	один порт для API, WebSocket, видео и файлов
Интерфейс	React 18, TypeScript 5.9, Vite 5, Ant Design 5 (`ru_RU`), TanStack Query 5, Zustand 4, react-konva, ECharts 5, hls.js, `@rjsf/antd` , MSW 2	SPA, редактор зон, графики, формы по JSON Schema, моки
Тесты	pytest, fakeredis	31 тест без Docker
Развёртывание	Docker Compose	вся система одной командой

Облачных API и языковых моделей в пути расчёта нет: внешний ресурс один — TPU стенда LinQ.

15. Ограничения и развитие

15.1. Ограничения

- **Три сценария из восьми задуманных.** Падение, бег, курение, перелезание, высота без привязи не реализованы (§1.3).
- **Общий TPU стенда — бутылочное горлышко.** С двумя моделями проходит 6,6 кадров/с на всё; при семи камерах это около 1 кадра/с на камеру (§10.5). Задержка на TPU зависит от нагрузки других команд.
- `yolov8n-pose` **плохо видит мелких и далёких людей** (2–3 из 7 на пешеходном ролике). Для камер под потолком нужна модель крупнее или дообучение.
- **Модель техники путает погрузчик и рохлю**, mAP50 0,28–0,53. Для сближения это не важно, для отчётности по типу техники — важно.
- **Событийные метрики не измерены** (§10.7).
- **Режимы** `remote` **и** `pytpu` **— заготовки:** на этом стенде Jupyter не видит `/dev/tpu*`, бэкенд `pytpu` не написан.
- **Воркеры на стенде живут, пока жив сервер Jupyter;** после его перезапуска — снова `start_workers.sh`.
- **В режиме** `broker` **страница «Производительность» не знает имён моделей** (плитка «Моделей: 0», в истории замеров — модель «?»): набор моделей задают воркеры, а их описание в ответ клиенту пока не передаётся.
- **Безопасность настроена для демо:** учётки инфраструктуры по умолчанию, бакет MinIO читается анонимно (ключи содержат UUID), Redis открыт на хосте с паролем (для режима A2 из замысла), `INGEST_TOKEN=change-me`. Для продакшена — секреты, подписанные ссылки, закрытые порты.
- **Без миграций базы:** таблицы создаются при старте. Для продакшена — Alembic.
- **Лицензии:** Ultralytics YOLO — AGPL-3.0; для продукта нужна коммерческая лицензия или замена на модели под Apache-2.0 (YOLOX, RTMDet/RTMPose) — архитектура от этого не меняется. Демо-ролики — права у авторов, только для локального показа.

15.2. Куда двигаться

1. **Событийная оценка на своих роликах:** 20–40 инсценировок с разметкой интервалов → precision, recall, ложные тревоги в час, задержка до тревоги. Тем же набором — калибровка квантования на кадрах целевой сцены.
2. **Модель позы крупнее** (YOLOv8s-pose) или дообучение на кадрах сверху — главный рычаг качества для всех сценариев.
3. **Сценарии на позе без новых моделей:** падение и неподвижность (критичный сценарий), бег, перелезание через ограждение.
4. **Пропускная способность TPU:** программы с batch 4, запуск модели техники через кадр, выделенное устройство вместо общего стенда.
5. **Дообучение по отметкам операторов:** кадры, помеченные «ложное», уже лежат в MinIO — это готовый набор трудных примеров.
6. **Расстояние в метрах:** гомография по четырём точкам на полу для камер, где это нужно.
7. **Продакшен-обвязка:** миграции, секреты, HTTPS, резервное копирование журнала, Kubernetes или edge-узлы рядом с камерами.

16. Приложения

А. Метрики Prometheus

Метрика	Сервис	Что меряет
`ingest_frames_total`, `ingest_fps`, `ingest_online`	ingest	кадры на анализ, фактический FPS, поток идёт
`ingest_decode_seconds`	ingest	декод и JPEG-кодирование кадра
`ingest_reconnects_total`, `ingest_clips_total{status}`	ingest	переподключения к источнику, клипы
`inference_seconds{stage, backend}`	inference	задержка по стадиям: `pre`, `infer`, `post`, `rpc`, `total`
`inference_frames_total{backend}`, `inference_fps`	inference	обработанные кадры, FPS за 10 с
`inference_dropped_total{reason}`	inference	пропущенные кадры: `stale`, `busy`, `expired`
`inference_errors_total{backend}`, `inference_fallback_active`	inference	ошибки, аварийный режим
`stream_lag_seconds{stream}`	inference, analytics	от кадра до обработки в очереди `frames` / `detections`
`analytics_step_seconds`, `analytics_tracks`	analytics	трекинг и правила на кадр, активные треки
`violations_total{camera, scenario, kind}`, `evidence_errors_total`	analytics	начала и окончания нарушений, ошибки сохранения кадров
`events_created_total{scenario, severity}`, `event_store_seconds`	events	события, от кадра до записи в журнал
`event_e2e_seconds`	notifications	от кадра, подтвердившего нарушение, до отправки в браузер
`alert_delay_seconds`	notifications	от начала нарушения до оповещения (с окном подтверждения)
`ws_connections`, `notifications_sent_total{channel, type}`	notifications	клиенты WebSocket, отправленные уведомления
`http_request_seconds{method, route, status}`	все	длительность HTTP-запросов

Б. Изображения этого документа

Изображение	Чем получено
`docs/diagrams/01…07-*.png`	`python docs/diagrams/build.py` (headless Chrome)
`docs/img/ui_*.png`	`python docs/make_screenshots.py` — работающий сервис и Grafana
`docs/img/chart_latency.png`	`python docs/make_charts.py` (matplotlib)

PDF-версия собирается командой `uv run --with markdown --with websockets python docs/make_pdf.py` (нужен установленный Chrome или Edge; в конвейере ничего из этого не используется).

В. Сопутствующие документы

- BRIEF.md и Краткое_описание.pdf — то же решение на пяти страницах;
- README.md — запуск, сервисы, контракты, демо-ролики;
- docs/ARCHITECTURE.md — замысел архитектуры, стек, источники данных, риски (шире реализованного);
- docs/plans/ — планы команды по ролям и общая шкала;
- backend/ml/README.md — модели и их подготовка к TPU;
- backend/ml/worker/README.md — воркер на стенде, протокол, замеры;
- frontend/README.md — экраны, роли, устройство интерфейса.

Г. Лицензии и данные

LOCO — public domain; COCO 2017 — CC BY 4.0 (аннотации), изображения — по условиям Flickr. Ultralytics YOLO и веса — AGPL-3.0. SDK LinQ — по условиям организаторов хакатона. Демо-ролики — права у авторов, используются только для локального показа, в репозиторий не входят.